Published: 11 June 2025
Online AM: 13 May 2025
Accepted: 29 April 2025
Revised: 15 April 2025
Received: 13 July 2024

SURVEY

Recent Advances in Symbolic Regression

JUNLAN DONG, South China University of Technology, Guangzhou, Guangdong, China

JINGHUI ZHONG, South China University of Technology, Guangzhou, Guangdong, China

Recent Advances in Symbolic Regression

JUNLAN DONG, South China University of Technology, Guangzhou, China
JINGHUI ZHONG, South China University of Technology, Guangzhou, China

Symbolic regression (SR) is an optimization problem that identifies the most suitable mathematical expression or model to fit the observed dataset. Over the past decade, SR has experienced rapid development due to its interpretability and broad applicability, leading to numerous algorithms for addressing SR problems and a steady increase in practical applications. Given the lack of a comprehensive review of the current literature on SR and its significance to both academia and industry, this article provides an in-depth overview of SR. The survey begins by outlining the background of SR and introducing it from three aspects: its definition, benchmarking datasets, and evaluation metrics. We also highlight the latest advancements in SR, summarizing the current research status. The review focuses on deterministic methods, genetic programming methods, and neural network methods, offering a thorough analysis of the advantages and limitations of various algorithms. Following this, key application scenarios of SR are introduced, and some commonly used software tools are summarized. Finally, the article provides an outlook on future research directions. This survey reviews the latest developments in SR and offers insightful guidance for readers who are new to the field.

CCS Concepts: • **General and reference** → **Surveys and overviews**; • **Computing methodologies** → *Modeling and simulation;*

Additional Key Words and Phrases: Symbolic regression, interpretable AI, genetic programming, neural network

ACM Reference Format:
Junlan Dong and Jinghui Zhong. 2025. Recent Advances in Symbolic Regression. *ACM Comput. Surv.* 57, 11, Article 290 (June 2025), 37 pages. https://doi.org/10.1145/3735634

1 Introduction

Symbolic regression aims at discovering mathematical expressions that describe the relationships between variables in the observed dataset [101, 183], which has been proved to be NP-hard [188, 208]. The computational resources required for solving SR are closely linked to the specific method category chosen. Compared to other machine learning algorithms, SR offers several distinct advantages. (1) No predefined model structure: Unlike traditional methods, SR does not rely on a fixed model form; (2) Interpretability: It can automatically generate concise, human-readable

This work was supported in part by the National Science Foundation of China under Grant 62476096, and in part by the Guangdong Basic and Applied Basic Research Foundation under Grant 2023A1515012291.
Author's Contact Information: Junlan Dong, South China University of Technology, Guangzhou, Guangdong, China; e-mail: eru-dd@foxmail.com; Jinghui Zhong (Corresponding author), South China University of Technology, Guangzhou, Guangdong, China; e-mail: jinghuizhong@gmail.com.

ACM 0360-0300/2025/06-ART290
https://doi.org/10.1145/3735634

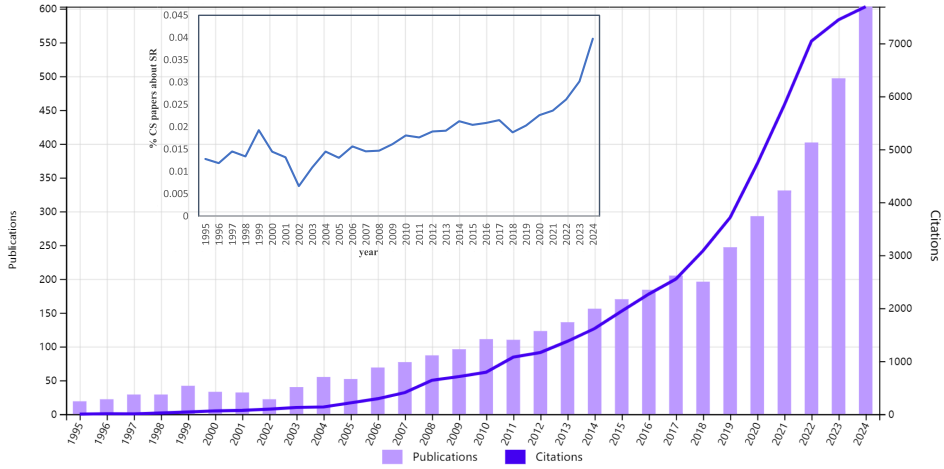

Fig. 1. Total publications and citations referring to SR by years from Web of Science until December 2024. The line chart shows the percentage of SR-related articles in CS by year.

mathematical expressions, facilitating a better understanding of model behavior and the underlying data relationships [100, 209]; (3) Strong extrapolation: SR often outperforms other algorithms in scenarios with limited training data, demonstrating superior extrapolation capabilities [215].

The origins of SR trace back to Kepler's trial-and-error approach [91] and Newton's laws of motion [158], which utilized symbolic models to fit experimental data. The term "symbolic regression" had not emerged in the 1970s. Sampson[181] introduced the concept of "adaptation" and explored genetic algorithms for adaptive modeling and automated search set the stage for its development. Building on this, Koza [103] formalized **Genetic Programming (GP)** in the 1980s, applying natural selection principles to evolve computer programs, which laid a critical groundwork for SR. The 1990s saw SR gaining recognition as an independent research field. Koza's GP was widely applied to function regression, marking the rise of SR [102], while Kinnear's comprehensive work on GP further advanced its theoretical framework [96]. Subsequent studies introduced innovations such as enhanced representations and improved genetic operators to boost SR's efficiency and versatility [10, 99, 165, 186]. In the 21st century, SR has reached a state of maturity, with its applications expanding across a wide range of domains. Notably, the pioneering work by Schmidt and Lipson on extracting natural laws using SR demonstrated its immense potential for scientific discovery, laying a strong foundation for its use in data-driven research [183]. In recent years, the integration of **neural networks (NNs)** into SR has garnered significant attention. NNs, with their robust nonlinear modeling capabilities, have addressed some inherent limitations of GP, thereby catalyzing further advancements in the field [19, 19, 73, 170]. The versatility of SR is evident in its successful deployment across diverse applications, such as uncovering insights in scientific research, enhancing software development and maintenance, and driving innovation in industrial design [95, 231].

Over the past few decades, SR has garnered growing attention within the research community. As shown in Figure 1, there has been a notable year-over-year increase in published SR articles in recent years, underscoring the sustained interest among researchers in this field. The prominence of SR can be attributed to its unique capability in generate potentially interpretable mathematical expressions. This ability allows SR to unveil underlying patterns in data, offering profound insights for scientists, engineers, and researchers.

Compared to the rapid development of SR, there is currently limited comprehensive literature reviewing the latest research progress on SR methods. Angelis et al. [8] conducted an initial

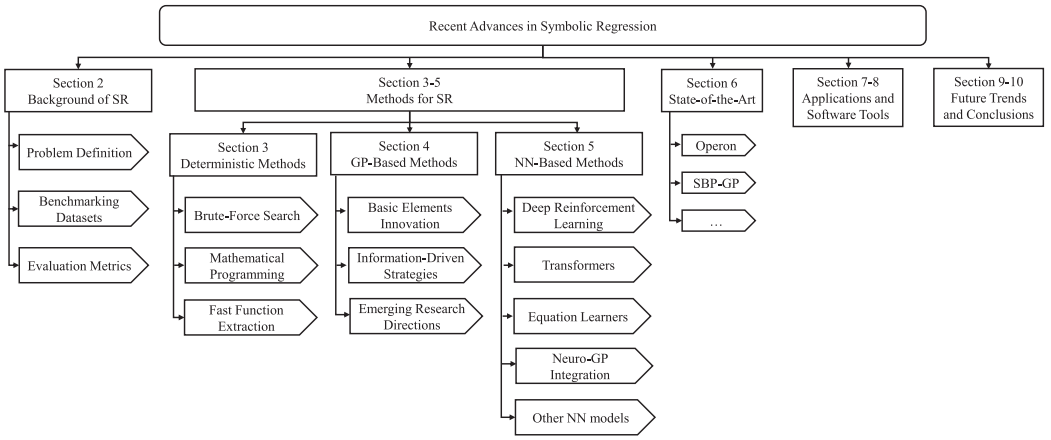

Fig. 2. Organization of the survey.

exploration of SR from a physics standpoint, emphasizing its practical applications. Makke et al. [136] discussed SR methods by categorizing them into linear and nonlinear approaches. However, these articles mainly focus on general concepts or specific techniques of SR. So far, no survey has summarized the latest advancements in SR, the commonly used tools, and the exciting subtopics within this field. Therefore, this review will comprehensively cover these aspects, aiming at filling this gap in the existing research. The contributions of this article are as follows:

— Organization of Resources and Practical References. This review summarizes commonly used datasets and evaluation metrics in SR. Providing these foundational resources enables researchers to grasp key concepts quickly and seamlessly transition into practical applications.
— Systematic Categorization and Methodological Review. SR methods are systematically classified into three major categories based on their core principles and technical approaches: deterministic methods, GP-based methods, and NN-based methods. This categorization reflects a paradigm shift in symbolic regression from rule-based and heuristic evolution to data-driven learning. Deterministic methods follow predefined rules or enumerative strategies; GP-based methods rely on stochastic symbolic search (e.g., mutation and crossover); in contrast, NN-based methods integrate structure and parameter learning within differentiable frameworks using gradient-based optimization. For each category, this review provides an in-depth analysis of representative works, applicable scenarios, and technical characteristics, offering a clear understanding of their strengths and limitations.
— Comprehensive Survey on Tools and Applications. This review compiles advanced SR methods and software tools, facilitating efficient research and application development. Additionally, it explores SR's popular applications in fields such as engineering and physical modeling and proposes potential future directions, offering a comprehensive view and guidance for further exploration.

The remainder of the article is organized as shown in Figure 2. Section 2 introduces the background, including problem definitions, benchmark datasets, and evaluation criteria. Sections 3–5 focus on methods: Section 3 details deterministic methods, Section 4 reviews GP-based approaches, and Section 5 discusses NN-based methods. Section 6 presents ten advanced methods currently in use. Section 7 covers representative SR applications, Section 8 summarizes common tools and software, Section 9 outlines future directions, and Section 10 concludes the article.

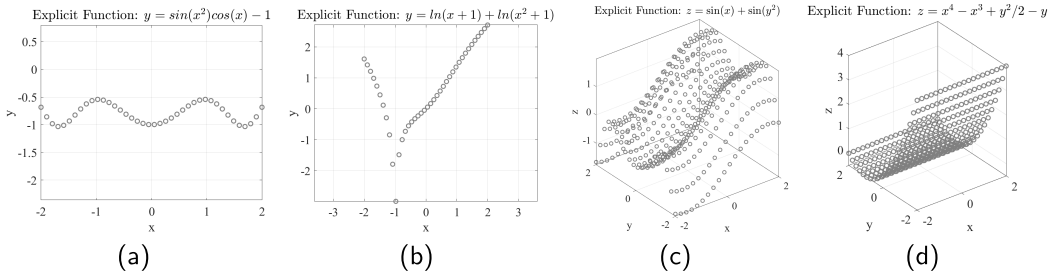

Fig. 3. Some examples of ESR.

2 Background of SR

This section offers the relevant background on SR to help readers understand its concepts and applications better. Additionally, it outlines commonly used SR datasets for training and evaluating the performance of SR models.

2.1 Problem Definition

SR aims at discovering mathematical expressions that accurately model the relationship between input and output variables, without assuming a predefined functional form. SR can be broadly categorized into two types: **Explicit Symbolic Regression** (**ESR**) and **Implicit Symbolic Regression** (**ISR**).

ESR: The goal of ESR is to identify an explicit functional form that represents the observed data, as illustrated in Figure 3. Give a dataset $D = \{\mathbf{x}_i, y_i\}_{i=1}^{N}$, N is the number of data observations; $\mathbf{x}_i \in \mathbb{R}^d$ is the input variables, $y_i \in \mathbb{R}$ is the output. The loss function of ESR can be described as

$$L(f) = \sum_{i=1}^{N} l(f(\mathbf{x}_i), y_i). \tag{1}$$

A common choice for loss functions is the squared variance between the output and the prediction, i.e., $L(f) = \sum_{i=1}^{N}(y_i - f(\mathbf{x}_i))^2$. Certainly, alternative evaluation criteria can be used based on specific needs, such as complexity, runtime, and so on. The optimization goal of ESR is to find the optimal explicit expression f^* over the set of functions that minimizes the loss function:

$$f^* = \arg\min_{f} L(f). \tag{2}$$

ISR: The goal of ISR is to identify an implicit functional form that represents the observed data, as illustrated in Figure 4. The loss function of ISR can be described as

$$H(F) = \sum_{i=1}^{N} h(F(\mathbf{X_{input}}), 0), \tag{3}$$

where $\mathbf{X_{input}} \in \mathbb{R}^d$ is the input vector that contains several variables from the observed dataset, N is the number of data observations. Searching for implicit functions is particularly challenging because error measures are difficult to define. At present, there are some preliminary explorations of work [37, 184]. The optimization goal of ISR is to find the optimal implicit expression F^* over the set of functions that minimizes the loss function:

$$F^* = \arg\min_{F} H(F). \tag{4}$$

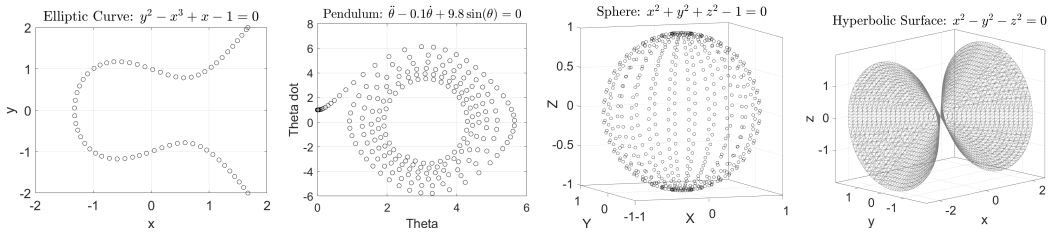

Fig. 4. Some examples of ISR.

Table 1. Benchmarking Datasets For SR

Datasets	Types	Problems	Applicability				Limitations			
			HO	PO	CO	ST	LV	LD	CF	HC
Koza [101]	Classic	3	✓	✓			✓	✓		
Nguyen [198]	Classic	12	✓	✓				✓		
Korns [99]	Classic	15		✓	✓				✓	
Keijzer [89]	Classic	15		✓	✓				✓	
Vladislavleva [209]	Classic	8		✓	✓				✓	
R [105]	Classic	3	✓				✓			
Jin [81]	Classic	6		✓	✓		✓		✓	
Livermore [170]	Classic	22	✓	✓	✓			✓		
AI Feynman [196]	Physical	120		✓	✓	✓				✓
SRSD [142]	Physical	240				✓				✓
Zhong [238]	Implicit	23			✓	✓	✓		✓	
Strogatz [190]	ODE	10				✓	✓			
PMLB [163]	Synthetic, Real-word	–								
UCI [11]	Synthetic, Real-word	–								
OpenML [202]	Synthetic, Real-word	–								
SRBench [117]	Synthetic, Real-word	252								
SRBench++ [52]	Synthetic, Real-word	252								

High Order problems (HO), Polynomial problems (PO), Coefficient optimization problems (CO), and Specific type problems (ST). "LV" indicates Lack of variety; "LD" indicates low dimension; "CF" indicates complex feature; "HC" indicates high computational cost;

ISR has stronger expressive power compared to ESR. Further, ISR simplifies the functional representation between variables. For example, an implicit function like $x^2 + y^2 - 1 = 0$ can describe a circle, while the explicit form would require two separate expressions $y = \pm\sqrt{1 - x^2}$. Consequently, searching for implicit functions is more challenging, as the relationships between the observed data are often more complex. We will introduce the research on ISR in the following Section 4.1.3.

2.2 Benchmarking Datasets

In this subsection, we present an overview of various datasets used to test SR, including classic SR datasets, physical datasets, and real-world datasets. Table 1 provides specific details for these datasets. At the same time, we also summarized the applicability and limitations of each dataset.

These classic datasets, primarily compiled by researchers including Koza, Nguyen, Korns, Keijzer, and Vladislavleva, were specifically designed for evaluating SR algorithms. Typically,

these datasets contain no more than five variables. For further details, refer to [144]. **AI Feynman dataset**[10] is generated from 100 equations selected from "The Feynman Lectures on Physics", covering classical mechanics, electromagnetism, quantum mechanics, and other physics topics. Additionally, the dataset includes 20 more challenging "bonus" equations extracted from other significant physics books. Further, **SRSD**[10] extended a dataset comprising 120 equations redesigned based on AI Feynman. **Strogatz dataset**[10] is an **Ordinary Differential Equation** (**ODE**) database, where each dataset represents a state of a two-state system described by a differential equation. Each dataset represents a natural process exhibiting chaotic and nonlinear dynamics. **Penn Machine Learning Benchmarks**, commonly known as Penn ML Benchmarks or PMLB[10], is a collection of benchmark datasets used for evaluating the performance of machine learning algorithms. **UCI datasets**[10], provided by the University of California, Irvine, constitute a diverse collection spanning multiple domains for machine learning research. **OpenML datasets**[10] are a collection of extensive, open machine learning datasets provided by the OpenML platform. This platform aims at serving as a centralized hub for sharing, discovering, and analyzing machine learning datasets. OpenML datasets cover various domains, including classification, regression, clustering, and tasks of different sizes and complexities. **SRBench**[10] includes 252 datasets from PMLB, covering both real-world and synthetic scenarios with or without ground-truth models. **SRBench++**[10] examines 252 datasets across two categories: Black-box Regression Problems (122 datasets) from PMLB without known ground-truth models, and Ground-truth Regression Problems (130 datasets) with known models, including those from the Feynman and ODE-Strogatz databases.

2.3 Evaluation Metrics

We have reviewed relevant literature and categorized the evaluation metrics for SR into three main aspects: **(1) accuracy**, **(2) interpretability**, and **(3) cost**.

For accuracy, commonly used metrics include **Mean Squared Error** (**MSE**), **root mean square error** (**RMSE**), and **normalized mean squared error** (**NMSE**), which are widely adopted in regression algorithms. Additionally, $\mathbf{R}^2$(**R-Square**) is frequently used in SR. Extrapolation capability is another key evaluation dimension, focusing on the final solution's performance on different datasets, especially unseen ones.

The evaluation of interpretability is less rigorous and standardized compared to accuracy [72, 223]. It generally includes the following aspects: solution simplicity, structural similarity, and dimensional homogeneity. Generally, the simpler the solution, the stronger its interpretability. Solution simplicity refers to the number of nodes, the depth of the expression tree, and the number of input variables. Dimensional homogeneity evaluation checks whether the units of the variables in the formulas generated by SR match and ensures dimensional balance in the formula, validating the model's physical or engineering applicability. Structural similarity measures the degree of alignment between the solution's structure and the expected patterns or known rules. This is crucial as it helps identify models that are not only accurate but also align with prior knowledge or expected system behavior. Finally, the evaluation can be performed by calculating the cost, which includes factors such as training time and resource consumption, offering a comprehensive assessment of the algorithm's efficiency and practicality. Table 2 summarizes the evaluation metrics of SR and gives some literature for readers' reference.

3 Deterministic Methods

The deterministic approach to SR aims at constructing mathematical expressions through systematic, non-random search strategies. This section mainly introduces three categories: brute force search, mathematical programming, and Fast Function Extraction.

Table 2. Evaluation Metrics Of SR

Evaluation Criterion	Sub-indicators	References
Accuracy	• MSE/RMSE/NRMSE/...	[3, 7, 16, 19, 36, 70, 81, 84, 128, 228, 237]
	• R^2	[52, 82, 101, 117, 156, 176, 192, 194, 215]
	• Extrapolation Ability	[52, 140, 179, 229]
Interpretability	• Simplicity	[23, 82, 123, 203, 219, 225]
	• Structural similarity	[71, 156]
	• Dimensional Homogeneity	[65, 92, 177, 178]
Cost	• Training Time	[34, 82, 93, 225]
	• Evaluation Time	[24, 180]
	• Resource Consumption	[20, 137]

3.1 Brute-force Search

Brute force search exhaustively explores all mathematical expressions to find optimal models. For example, **Priority Grammar Enumeration** (**PGE**) [217] uses grammar rules to guide the search from simple to complex equations, while AI Feynman [196] applies brute force to derive physics subexpressions. Although it ensures global optimality, its computational cost grows exponentially with variable dimensions. To address this, Kronberger et al. [109] constrained syntax by generating fixed-length models and clustering them, and Kammerer et al. [84] combined context-free grammar with heuristic optimization. Bartlett et al. [16] introduced Exhaustive SR, which systematically explores equations within set constraints and leverages parallelization for efficiency.

3.2 Mathematical Programming

In SR, **mixed-integer programming** (**MIP**) [14] is a commonly used deterministic method, but its computational efficiency is low. To improve efficiency, Austel et al. [13] combined mixed-integer nonlinear optimization with explicit enumeration and dimensional analysis constraints. Kim et al. [93] introduced new cuts for **mixed-integer nonlinear programs** (**MINLPs**) and developed a heuristic for iteratively constructing expression trees. Neumann et al. [157] enhanced MINLP efficiency by representing equations as directed acyclic graphs, reducing binary variables. Engle and Sahinidis [64] proposed a deterministic MINLP formulation, leveraging auxiliary expression trees and the chain rule for efficient derivative computation.

3.3 Fast Function Extraction

Fast Function Extraction (**FFX**) [143] is a deterministic method using elastic-net regression for SR, known for its efficiency and scalability. Kammerer et al. [85] enhanced FFX with **Nonlinear Least Squares** (**NLS**) optimization and a variable projection algorithm, yielding simpler models. Vaddireddy and San [200] applied FFX to discover **partial differential equations** (**PDEs**), using compressive sensing and sparse optimization for feature selection and parameter estimation, generating multiple models and optimizing complexity, and accuracy through non-dominated sorting.

3.4 Summary

Deterministic methods in SR offer significant advantages, including ensuring the discovery of globally optimal solutions, generating mathematically interpretable expressions, and demonstrating high stability and reliability. However, these methods face high computational complexity, particularly as problem complexity and dimensionality increase, leading to search space explosion, long computation times, and reliance on specific problem structures. Potential improvements include

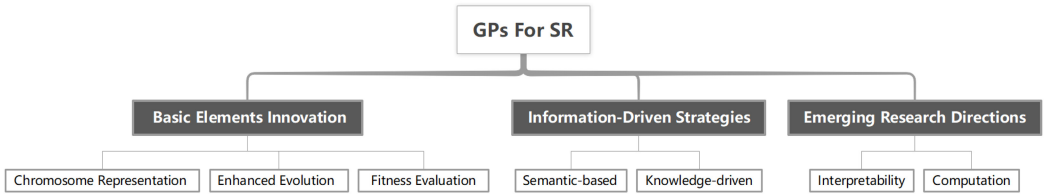

Fig. 5. The summary map of GPs.

combining deterministic methods with heuristic algorithms to reduce computational costs while retaining accuracy, introducing constraints or prioritization rules to optimize search efficiency, and utilizing modern parallel computing and distributed systems to accelerate evaluating of complex problems.

4 GP-based Methods

The research directions of GP-based methods mainly cover three key areas, as shown in Figure 5. First, innovation in the core elements of GP, including novel chromosome representations, optimization operator designs, and improvements to fitness functions. Second, the exploration of strategies that utilize additional information, such as semantic-driven and knowledge-driven GP methods, to further enhance algorithm efficiency and performance. Lastly, emerging research focuses on improving interpretability and utilizing parallel computing techniques.

4.1 Basic Elements Innovation

GP is an evolutionary algorithm that evolves a population of candidate programs or expressions to solve a given task. It starts with a randomly generated population and iteratively applies selection, crossover, and mutation based on a fitness function. Key components such as chromosome representation, fitness evaluation, and genetic operators jointly determine the search process and performance. The following sections describe these elements in detail.

4.1.1 Chromosome Representation. Chromosome representation is critical in GP, directly affecting its capability to model and optimize problems. Common encoding methods include tree-based[235, 237], linear[76], graph-based[138], and grammar-based representations[134]. Figure 6 illustrates the chromosome representation of four types.

- Tree-based encoding has a simple structure and is suitable for recursive structures and expression modeling. It is commonly used in SR, classification problems, and computer algebra systems that require hierarchical representations.
- Linear encoding is concise and efficient. Linear representation is suited for describing program flows and sequential operations, commonly used in optimizing control flows and state machines.
- Graph-based encoding is flexible and powerful, capable of handling complex relationships and structures. Graph-based encoding excels in circuit design and signal processing, effectively representing complex dependencies and shared sub-expressions.
- Grammar-based encoding generates solutions that conform to grammatical rules, making it suitable for representing complex models with constraints, often used in program synthesis and natural language processing tasks.

As the complexity of the problem increases, traditional encoding methods may struggle to handle more complex tasks. Therefore, more flexible and efficient encoding schemes can be designed to meet the diverse needs of different problems. Moreover, existing encoding methods may suffer

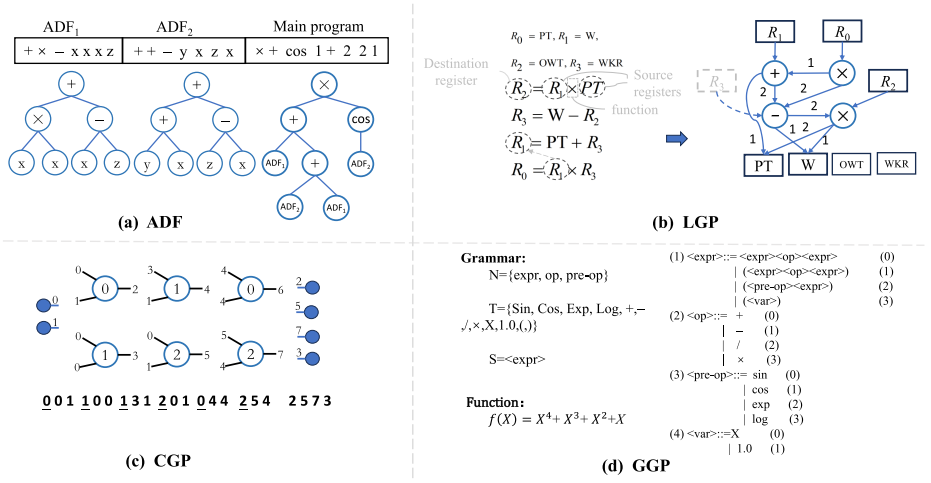

Fig. 6. Examples of four GP chromosome representations. "ADF" indicts Automatically Defined Functions based on tree. "LGP" indicts Linear Genetic Programming. "CGP" indicts Cartesian Genetic Programming. "GGP" indicts Cartesian Genetic Programming.

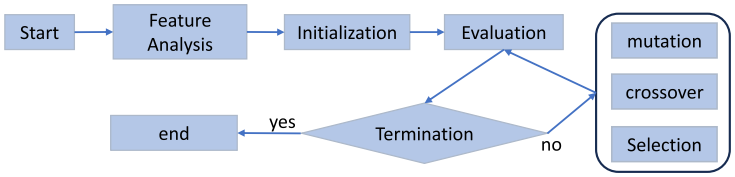

Fig. 7. A general process of GP.

from redundancy, leading to duplicate individuals in the search space and reducing population diversity. ADF help alleviate this issue by allowing reusable subcomponents, but research on ADF is still limited. Future work can focus on the design, optimization, and integration of ADF with other algorithms to further enhance the efficiency and scalability of GP.

4.1.2 Enhanced Evolution. GP is an evolutionary computation algorithm inspired by natural selection and genetics, as shown in Figure 7. It mimics evolutionary processes to iteratively refine randomly generated programs into solutions tailored to specific problems through selection, crossover, and mutation [172].

Feature analysis plays a vital role in enhancing GP model performance and computational efficiency. Researchers have developed various optimization methods, such as Al-Helali et al.'s genetic planning with feature importance thresholds [3] and Chen et al.'s two-stage feature selection strategy combining significant candidates with feature permutation [32, 36]. Moreover, dimensionality reduction techniques are also employed to address SR problems[239].

The optimization of genetic operators mainly involves crossover, mutation, and selection. Semantic crossover has emerged as a promising direction, as it mitigates the risk of disrupting semantic coherence by preserving meaningful gene combinations and aligning offspring with problem structures[125, 199]. Efficient mutation operators further enhance GP's search space exploration capability. For example, Al-Helali et al. prioritize mutation of poorly constructed trees to increase diversity [2], Virgolin and Bosman introduced coefficient mutation to optimize SR coefficients [207], and Kushida et al. proposed module mutation for both discontinuous and smooth functions

[115]. Recently, Lexicase Selection has garnered significant attention by evaluating individuals across multiple test cases to handle multidimensional fitness, thereby enhancing diversity and adaptability. Subsequent improvements include weighting test cases by importance [56], epsilon-lexicase selection [119], and feature-based probabilistic methods [57]. Finally, adaptive operators are an essential tool that dynamically adjusts evolutionary operations to balance global exploration and local exploitation. For example, a dual-tree adaptive mutation mechanism using probability vectors has been proposed [224].

4.1.3 Fitness Evaluation. **(1) Accuracy evaluation.** In the innovations of fitness evaluation, accuracy evaluation extends beyond commonly used regression metrics (i.e., RMSE, MSE, R^2). Since error metrics such as MSE, RMSE, and NMSE have simple mathematical relationships and their variations are monotonic, these metrics may yield identical rankings when comparing the performance of individuals. This could potentially impact the selection process and the effectiveness of evolution. To address this issue, some researchers incorporate constraint conditions on top of accuracy requirements to address more complex environments. For example, Kubalik et al. [113] considered both training accuracy and adherence to prior knowledge about desired model properties during the optimization process. Haider et al. [74] extended multi-objective algorithms with shape constraints, employing a soft penalty approach to minimize constraint violations and prediction errors during the model training process. Furthermore, some researchers have developed population diversity as a metric to help GP achieve a good balance between exploration and exploitation [25, 35].

(2) Balancing Accuracy and Interpretability. As black-box models like deep learning become more prevalent, the focus on interpretability, especially in GP, has grown. By incorporating interpretability into fitness evaluation, GP enhances both accuracy and transparency in its learning process. However, achieving the right balance between interpretability and accuracy remains a key challenge. One effective enhancement is integrating regularization constraints into the fitness function. For example, Vanneschi and Castellia [203] introduced soft-target regularization to assess functional complexity, mitigating overfitting. Ni and Rockett [161] applied Tikhonov regularization, measuring semantic complexity, to multi-objective GP, improving performance. Additionally, Montaña et al. [150] proposed a regularized fitness function based on Vapnik Chervonenkis dimension upper bounds, aiding GP in addressing noisy SR problems. Another type of multi-objective evaluation mechanism involves multiple objective functions such as goodness of fit, model complexity, and interpretability, rather than focusing on a single objective. Kommenda et al. [98] used predictive accuracy and complexity of the model as fitness functions for multi-objective optimization. Furthermore, Rafiq et al. [174] incorporated hierarchy in optimization, initially using a single-objective fitness function and introducing complexity as a new objective under specific conditions before transitioning to multi-objective training.

(3) Implicit function evaluation. Implicit function evaluation does not directly provide a clear fitness value but rather measures the model's effectiveness through implicit relationships or conditions. The classical evaluation method involves computing partial derivatives of a potential implicit equation and utilizing the average logarithmic error of these derivatives for fitness assessment [184]. The specific computational formula is as follows:

$$-\frac{1}{N}\sum_{i=1}^{N} log\left(1+|\frac{\Delta x_i}{\Delta y_i}-\frac{\partial x_i}{\partial y_i}|\right), \tag{5}$$

where N is the number of observed data, $\frac{\Delta x}{\Delta y}$ is an implicit derivative estimated from the data, and $\frac{\partial x}{\partial y}$ is the implicit derivative derived from the candidate implicit function. Subsequently, Pan

et al. [166] improved the DB-FEM by introducing hyper-parameters to the constraint term of the derivative, thus shifting the derivative away from the constant of zero. Furthermore, Chen et al. [37] proposed a novel method called **Comprehensive Learning Fitness Evaluation Mechanism (CL-FEM)** for evaluating relationships in implicit functions. Unlike derivative evaluation mechanisms, CL-FEM can learn from interference information in stochastic datasets S to validate implicit equations, thereby reducing complexity and addressing sparse data problems. The specific evaluation formula is shown below:

$$V = \begin{cases} MSE, & if \left(m_1 > 10^{-4} \wedge m_2 > 10^{-4} \wedge \cdots \wedge m_k > 10^{-4}\right) = TRUE \\ 10^{10}, & otherwise \end{cases}, \tag{6}$$

where

$$m_k = \frac{1}{N} \sum_{i=1}^{N} \left(S_i - S_i^k\right)^2, \ k = 1, 2, \ldots, D. \tag{7}$$

If $m_k > 10^{-4}$, it indicates that the candidate model is meaningful in the kth dimension. The MSE is computed as the fitness value for the candidate model only when all m_k are greater than 10^{-4}; otherwise, it is assigned an infinite value. For a detailed description of the algorithm, please refer to the [238]. Moreover, Wen et al. [211] proposed a new evaluation mechanism by applying the Bolzano theorem to approximate the solutions of implicit equations, achieving promising results.

Evaluating implicit functions is more challenging than explicit ones, making the development of effective fitness evaluation methods for implicit functions a promising research direction.

4.2 Information-driven Strategies

4.2.1 Semantic-based Approaches. In recent years, integrating semantic information into GP has gained attention, focusing on aspects such as population initialization, genetic operators, and defining new ones. These methods aim at accelerating convergence, maintain semantic diversity, study semantic locality, and expand semantic-related features.

Indirect SGP utilizes semantic knowledge to guide the GP search, aiming at accelerating convergence and reduce the search space. It leverages semantic information in three ways [204]: ensuring semantic behavioral diversity to enhance population diversity [17, 159], guiding the search toward adjacent semantic regions of the best individual for smoother convergence [171, 199], and applying geometric information in crossover operations to improve algorithm performance [104, 105].

Direct SGP involves defining specific semantic operators or mutation operators designed to enhance the search efficiency in GP. These operators leverage the semantic information of symbols to guide the evolutionary direction of programming. For instance, Jeong et al. [80] introduced a semantic clustering operator to improve GP convergence, while Huang et al. [77] designed a linear chromosome representation and semantic genetic operators to propagate semantic errors in programs. Noteworthy work includes **Geometric Semantic Genetic Programming (GSGP)** [151], which directly searches the semantic space of functions using semantic-aware crossover and mutation operators to improve search efficiency. Pawlak showed that GSGP programs are random linear combinations of partial solutions and can be constructed more simply within a set timeframe, guaranteeing optimal solutions to inductive problems [168]. However, GSGP focuses on accuracy fitting rather than symbolic fitting, often resulting in significant tree bloat. The subsequent development of various methods has aimed at improving GSGP, with some focusing on accelerating the convergence process [162], others on controlling the size of offspring [139, 205], and still others on enhancing generalization performance [30, 33].

Semantic-based GPs face challenges such as high computational complexity and over-reliance on semantic information. To tackle these challenges, approaches such as integrating multi-level

semantics, varying mutation operations, and enhancing computational efficiency can be explored.

4.2.2 Knowledge-driven Approaches. Knowledge-driven SR utilizes domain-specific knowledge or prior information to guide modeling and prediction. Common approaches in GP include initializing the population with prior knowledge to speed up convergence [167], adjusting the fitness function to favor solutions aligned with this knowledge, and modifying the search direction to find specific solutions [92, 133]. Furthermore, constraints based on prior knowledge can be introduced to ensure solutions comply with known problem features, such as guiding model training with formal constraints like monotonicity and symmetry [112], or constraining function values and derivatives [108]. Apart from that, transfer learning has also been employed to improve GP search efficiency, with frameworks like Chen et al.'s [34] transfer learning approach using differential evolution to optimize weights and accelerate the search, and zhong et al.'s [234] knowledge transfer mechanism for multi-form GP. Moreover, Munoz et al. [154] demonstrated that out-of-domain knowledge can help GP solve real-world problems across domains, and Al-Helali et al. [2] introduced multi-tree GP for transfer learning, improving missing value estimation. Additionally, surrogate models have been employed to aid the search process, enhancing the efficiency of handling SR tasks [88, 222]. Not only these, several studies have also integrated subject-specific domain knowledge into the SR search process [69, 191].

Knowledge-driven GP improves search efficiency and solution quality but faces challenges like reliance on high-quality domain knowledge, integration complexity, and reduced diversity. Future advancements could focus on automated knowledge acquisition, dynamic updates, and integrating diverse knowledge sources (e.g., semantics, statistics, physical constraints) to enhance robustness and applicability.

4.3 Emerging Research Directions

4.3.1 Interpretability Improvement. Interpretability is key for understanding model behavior, explaining decisions, and building trust. Unlike black-box models like NNs, GPs are inherently interpretable. Mei et al. [147] review research on interpretable AI using GP, focusing on intrinsic and post-hoc interpretability. However, complex symbolic structures in many GP models still hinder comprehension. To improve this, strategies like structural simplification, feature importance analysis, and visualization tools have been explored.

Simplifying the generated structure is a key approach to enhancing interpretability in GP. This involves applying heuristic rules or structural constraints to produce concise and comprehensible symbolic representations. Common strategies include limiting tree depth [206], employing bloat control operators [49], and pruning redundant components [27]. Additionally, replacing complex tree structures with simpler, semantically meaningful alternatives has gained attention [39]. Noman et al. [79] provide a comprehensive overview of these simplification techniques.

Feature importance analysis enhances interpretability by evaluating each feature's impact on the model's output through monitoring usage, analyzing coefficients, and assessing program structures. For example, Aldeia et al. [5] used partial effects to estimate factor importance in the ITEA algorithm, aligning with ground-truth explanations. Miranda et al. [148] applied local linear regression to analyze input attribute impacts, while Dick et al. [55] normalized features to ensure equal contributions to predictions.

To further enhance interpretability and generalizability, researchers have also tackled the issue of dimensional consistency in solutions. Ensuring consistent units of measurement prevents interpretational conflicts and improves practical applicability. Montana [149] introduced **Strongly Typed GP** (**STGP**), which enforces unit consistency across all solutions. Keijzer and Babovic [90]

extended this idea with an enhanced GP approach, balancing unit consistency with challenges such as reduced accuracy and increased complexity.

Visualization techniques enhance interpretability by presenting solutions in intuitive formats. These methods convert symbolic structures into graphical representations, simplifying the observation of algorithmic processes and outcomes. For example, Medvet et al. [146] used diversity and DU graphs to visualize population diversity and gene contributions, while Nguyen et al. [160] applied dimensionality reduction for analyzing GP performance. Lensen et al. [126] introduced GPtSNE, a method that evolves interpretable dataset mappings into high-quality visualizations for better understanding of GP outputs.

4.3.2 Parallel and Distributed. GP is computationally intensive, especially for complex and large-scale SR problems, often resulting in slow operation. To address this, researchers have developed distributed and parallel computing approaches to improve efficiency. Distributed GP distributes tasks across multiple nodes, focusing on population distribution, where individuals are allocated to different processors, as demonstrated by Fernández et al.[67] with multi-population parallel GP, and Huang et al. [78] with a hierarchical parallel computing framework. It also includes dimension distribution, where problem dimensions are assigned to computing nodes, such as in Klein et al.'s AJAX-based system for distributed fitness evaluations [97], and Dong et al.'s federated GP framework for secure distributed SR [59]. Parallel GP, on the other hand, accelerates GP by handling tasks concurrently, primarily focusing on parallelizing fitness evaluations. Zhang et al. [230] utilized GPU parallelism with a CUDA-based SR algorithm, Augusto et al. [12] leveraged OpenCL for evaluation speedup, and Burlacu et al. [24] proposed the Operon C++ framework for task-level parallelism. Additionally, Zeng et al. [226] evaluated parallel GP scalability and runtime performance, while Sambo et al. [180] introduced **Asynchronous Parallel GP** (**APGP**) to reduce complexity during evolution.

In summary, distributed GP distributes computational tasks across nodes for global optimization, while parallel GP enhances local computational efficiency by executing tasks concurrently within a single node. Future work could focus on developing a universal, easy-to-use, and high-performance open-source GP platform.

4.4 Summary

Table 3 summarizes the GP methods discussed in this article along with their code. As a classical SR approach, GP has shown strong accuracy and wide applicability across domains. Its diverse chromosome representations provide rich expressiveness and adaptability, enabling the flexible construction of complex models for various regression tasks. Unlike black-box models, GP offers high interpretability by clearly revealing the solution structure and underlying logic, making it well-suited for tasks requiring model transparency. Despite these strengths, GP has several limitations. Its large search space can lead to overfitting, particularly with limited or noisy data. Performance is also sensitive to hyperparameter settings, with improper choices potentially causing instability or long training times. While effective on low-dimensional problems, GP struggles with high-dimensional data due to increased computational complexity, feature redundancy, and the curse of dimensionality. Additionally, GP may get trapped in local optima, affecting final model quality. To maximize its effectiveness, especially in high-dimensional settings, it is important to fine-tune hyperparameters and apply techniques like feature selection or dimensionality reduction to control complexity and improve performance.

5 NN-based Methods

In the past several years, using NNs to address SR problems has become a growing trend. These approaches include **deep reinforcement learning** (**DRL**), transformers, **equation learners** (**EQL**),

Table 3. GP Methods For SR

Method	Description	Year	Code
	Basic Elements		
GE [164]	Grammatical Evolution, Backus-Naur Form	2001	App. 10
SL-GEP [237]	Tree representation, ADF	2016	App. 10
Ve-GP [15]	Vector-based representation	2019	– –
GB-LGP [15]	Linear GP, Stochastic Context-Free Grammar	2017	– –
MLSI [76]	Linear representation, Directed acyclic graph	2023	App. 10
DLS_GP [227]	Double-Stage Lexicase Selection Operator	2023	App. 10
GGPES [152]	Hybrid Technique, Grammar GP, Evolution Strategy	2018	– –
PLS [57]	ϵ-plexicase selection	2023	App. 10
Ref [107]	Grammatical evolution, Population diversity	2020	– –
Ref [7]	Grammar Pruning, Effective Genome Length	2021	– –
Ref [36]	Feature Selection, High-Dimensional SR	2017	– –
Ref [3]	Feature Removal Impact, High-Dimensional SR	2024	– –
MTGP [4]	Multitree GP, Incomplete Data	2023	– –
	Information-Driven Strategies		
MSGP [125]	Memetic Semantic	2023	App. 10
GSGP [151]	Semantic Geometric Operators, Semantic Fitness Landscapes	2012	App. 10
GPALGX [31]	Angle-aware Mating Scheme	2017	– –
ADGSGP [33]	Angle-Awareness, Geometric Semantic Operator	2018	– –
GSGP-Red [139]	Solution Size, Function Simplification	2018	App. 10
SBP-GP [205]	Semantic Back-Propagation utilized in GP	2019	App. 10
SLGP [77]	New Chromosome Representation, Mutate-and-divide Propagation	2024	App. 10
GP/TS-S [40]	Tournament Selection,Statistical Test	2018	App. 10
pMTGPDA [2]	Transfer Learning, Domain Knowledge	2021	– –
Ref [112]	Prior Knowledge, Multi-Objective SR	2020	– –
Ref [108]	Prior Knowledge, Shape-Constrained SR	2022	– –
SciMED [92]	Domain Knowledge, Physics-Informed SR	2022	App. 10
	Emerging Research Directions		
ELA [148]	Post-Hoc Interpretability	2020	App. 10
PE-ITEA [5]	Feature Importance, ITEA	2021	– –
DU map [146]	Diversity and Usage map, Visualization	2018	App. 10
APGP [180]	Asynchronous Parallel Computing	2021	– –
GPOCL [12]	OpenCL, GP-GPU	2013	– –
Ref [226]	Platforms Comparison, MPI, GPU, Spark	2020	– –
HSL-GEP [78]	Adaptive-weight, Multi-population, GPU	2020	– –

"MLSI" indicts Multitask LGP with Shared Individuals. "PLS" indicts Probabilistic Lexicase Selection. "ELA" indicts Explanation by Local Approximation. "ITEA" indicts Interaction-Transformation Evolutionary Algorithm. "PE-ITEA" indicts ITEA coupled with Partial Effect. "APGP" indicts asynchronous parallel GP. "HSL-GEP" indicts high-performance self-learning gene expression programming algorithm. "GB-LGP" indicts Grammar-Based LGP. "GPALGX" indicts GP implements the angle-awareness into one recent approximate geometric crossover.

hybrid methods that integrate NN with GP (Neuro-GP integration), and other NN models such as the increasingly popular **Kolmogorov-Arnold Networks** (**KANs**).

5.1 Deep Reinforcement Learning

DRL approximates mathematical expressions by learning complex strategies and action sequences. The key of deep reinforcement learning lies in the agent learning the optimal strategy through interaction with the environment, as shown in Figure 8. **State (*S*)** represents the observed data point

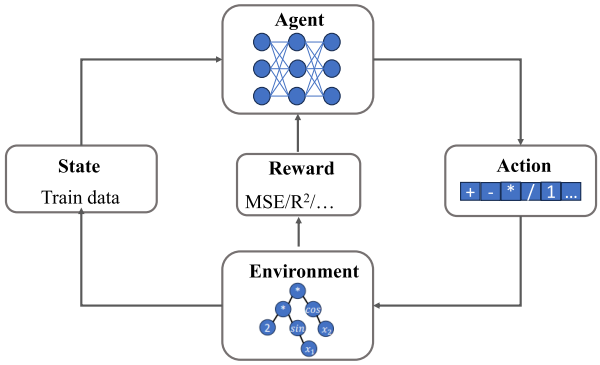

Fig. 8. A sample sketch of a general SR method based on DRL.

and the current tree structure; **Action (*A*)** corresponds to an expression that combines operators (e.g., +, -, *, /) or operands (i.e., variables, constants). **Reward (*R*)** evaluates the fit of the expression to the target function (i.e., MSE, R2). **Q-value function (*Q*(*S*, *A*))** predicts the expected cumulative reward for taking action A in state S as following:

$$Q(S,A) = \mathbb{E}\Big[\, R(S,A) + \gamma \max_{A'} Q(S',A') \Big], \tag{8}$$

where $\mathbb{E}$ is the expected value. γ is a discount factor that weighs the importance of a present reward against a future reward. S' is the new state after action A is performed.

In DRL methods, a noteworthy work is **Deep Symbolic Regression** (**DSR**) [170], which employs a novel risk-seeking policy gradient algorithm for training. In this approach, sampling tokens correspond to actions, while parent and sibling inputs act as observations, and the reward function is based on NRMSE. However, this method does not allow free parameters in the generated expressions, which significantly limits its functionality. To address this issue, Landajuela et al. proposed uDSR [122].

Building on the uDSR framework, some researchers have introduced physical constraints to restrict the freedom of the equation generator further, thereby enhancing performance [192]. Recently, Guo et al. [73] proposed an efficient gradient-based learning method called **Efficient Symbolic Policy Learning** (**ESPL**), which learns symbolic policies from scratch in an end-to-end differentiable manner. Some studies have also introduced grammatical knowledge to enhance search capabilities by guiding the search process. For example, Crochepierre et al. [47] proposed the **Reinforcement-Based Grammar-Guided Symbolic Regression** (**RBG2-SR**) method, which uses context-free grammar as the action space in reinforcement learning to constrain the representation space with domain knowledge. Building on this idea, they also developed an interactive platform [46] that generates expressions based on human preferences. Similarly, another notable advancement is a real-time search demonstration system for DSR, which enables users to interactively guide the search process by dynamically adjusting hyperparameters and manually selecting expressions during training [94].

Additionally, combining RL with GP is another promising direction. Zhang et al. [228] combined GEP and multi-agent RL for SR. They defined the state as a constant value and designed different action spaces: functional primitives for the head of the symbolic expression and terminal primitives for the tail. The reward was set to MSE. Mundhenk et al. [153] proposed a neural-guided component where RNNs generate samples for initial populations in GP. The GP-optimized samples are then combined with the initial populations of RNNs, allowing RNNs to gradually learn better starting populations.

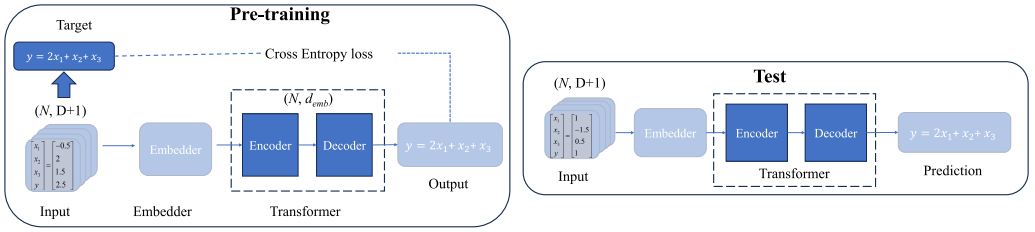

Fig. 9. An example of solving SR with transformer. Input represents data where D is the dimension of variables x and N is the observation size. The embedder encodes tokens into vectors with semantic or structural meaning. The encoder captures variable relationships and expression hierarchy, including priorities. The decoder generates symbolic expressions step-by-step, ensuring mathematical validity.

5.2 Transformers

The pre-trained transformer models have demonstrated significant potential in SR. Figure 9 is shown an example of solving SR with transformer.

Transformer models represent expressions as sequences [19, 121]. Unlike RL-based methods, they leverage large-scale pre-training to improve efficiency and inference speed [201]. These models construct expression frameworks [20] while simultaneously optimizing structures and coefficients [62]. For instance, Biggio et al. [20] utilized beam search and sampling methods as decoding strategies, generating multiple candidate equations from a given dataset. The best candidate is then selected based on fitting accuracy, with constants refined using the BFGS optimization algorithm. Similarly, Holt et al. [75] proposed the DGSR framework, which employs pre-trained deep generative models to capture the inherent patterns in equations, thereby improving optimization efficiency. However, the absence of a search mechanism in the DGSR model limits its ability to make targeted improvements for specific problems. To address this, Kamienny et al. [83] introduced Monte Carlo search, effectively mitigating this limitation.

In addition, from the computer vision perspective, Xing et al. [220] propose a DeSTrOI framework that aims at predicting the importance of each mathematical operator in order to reduce the search space for SR tasks. Li et al. [128] proposed to convert datasets into images and then generate expressions from the images.

The latest research trend is to integrate knowledge into the search process of transformer. Li et al. [130] used supervised contrastive learning to align features of expressions with the same skeleton. TPSR [187] incorporated non-differentiable feedback as external knowledge to improve extrapolation and noise robustness. Biggio et al. [21] injected prior knowledge to bias predictions toward structurally valid expressions, narrowing the search space. Additionally, transformer-based SR has been extended to dynamical systems, enabling the discovery of arbitrary nonlinear ODEs from multidimensional data without prior assumptions [51].

5.3 Equation Learners

EQL [140] employ specialized NN architectures to automatically discover mathematical expressions. By using symbolic operations from SR as activation functions and training with sparse constraints through backpropagation and gradient descent, EQL achieves interpretable results. This unique design transforms NN regression into a transparent and explainable process, as illustrated in Figure 10.

Unlike traditional NNs, the EQL network uses a set of primitive functions as activation functions. These activation functions are primitive operators that the final expression may include unary operators (e.g., cos, sin, ln) and binary operators (e.g., $+$, $-$, $\times$, $\div$). The i^{th} layer values can be

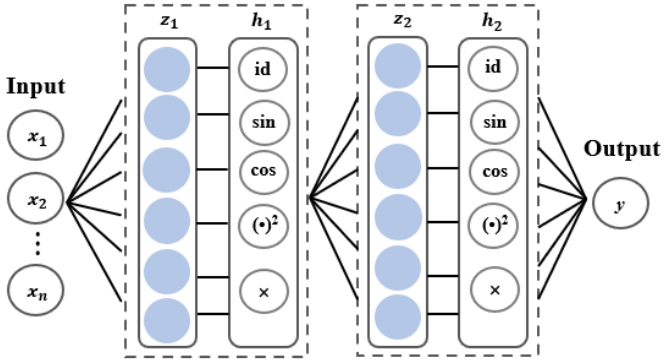

Fig. 10. An example of an EQL network architecture.

described as

$$z^i = W^i h^{i-1} + b^i, \tag{9}$$

$$h^i = f(z^i), \tag{10}$$

where h^0 is the input data, W^i is the weight matrix of the i layer, b^i is the bias of the i layer. $f \in \{id, sin, cos, sigm, \times\}$ represents function operators. z^i represents intermediate representation of the i layer. There are several approaches proposed in the literature to enhance EQL. These enhancements can be categorized into three paradigms: optimizing the network architecture, refining the learning algorithms, and introducing new regularization techniques.

In [179], an extended network architecture named EQL$^\div$ was introduced. EQL$^\div$ includes a division unit in its final layer, enabling it to model division within network structures. This adaptation is particularly suited for physical systems. Costa et al. enhanced EQL with OccamNet, a fast neural model for SR, described in [42]. OccamNet uses NNs to simulate function probability distributions, employs hopping connections for compact solutions, and optimizes via a new two-step gradient descent and evolutionary strategy training. Zhou and Pan proposed MathONet in [240], combining **Polynomial-Network** (**PolyNet**) and **Operation-Network** (**OperNet**) using Bayesian methods to find sparse subgraphs of NNs for solving ordinary and partial differential equations. Wu et al. introduced DeepSymNet in [219], simplifying the SR problem into two sub-problems. Dong et al. [60] proposed the **evolving Equation Learner** (**eEQL**), which introduces a unique network architecture based on ADFs. This novel design allows for dynamic adaptations of the network structure. Finally, [229] extends EQL with SEQL and PEQL networks, demonstrating strong generalization across analytical equations, partial differential equations, and high-dimensional datasets.

Moreover, several advancements have been pursued to improve the learning algorithms of EQL. Werner et al. [212] introduced **informed EQL** (**iEQL**), integrating expert knowledge of equation components and domain-specific sparse priors to produce interpretable models with strong predictive performance. Chen [29] proposed **Gumbel-Max equation learner** (**GMEQL**), employing a continuous relaxation approach and trainable structural and regression parameters. Kubal'ık et al. [114] presented N4SR, enhancing EQL with skip connections and adaptive optimization strategies for managing training, regularization, and constraint errors. Wu et al. [218] developed a greedy pruning algorithm for Equation Learners, preserving data fitting accuracy by selectively pruning the network and employing beam search for optimal solutions.

Apart from that, Kim et al. [95] improved EQL using $L_{0.5}$ sparse regularization and recommended multiple NN for kinematic SR, each trained on specific time slots. It has been reported outperform EQL on MNIST and dynamical systems.

5.4 Neuro-GP Integration

The combination of GP and NN leverages the strengths of both, providing more powerful solutions. For instance, the expressions generated by GP can be used as input for NN, or the output of the NN can be used as the optimization objective for GP [153, 236]. This combination can enhance the robustness and efficiency of the search, accelerate the convergence speed of the model, and enable the search to more effectively explore and utilize the structure and features of the problem. Further, some studies have explored using expressions generated by GP as activation functions for NN, optimizing them to enhance model performance [60]. Another work proposed a neural-encoded expression programming framework, consisting of an encoder and decoder, used to evaluate the fitness values of the evolutionary algorithm. The encoder determines the positions of symbols, while the decoder decodes strings into GEP expressions[9]. Furthermore, Razavi et al. [176] introduced a new GP framework called **functional algebraic genetic programming** (**faiGP**). This approach uses a variational autoencoder with two convolutional layers (ConVAE) for dimensionality reduction and a Bayesian classifier to generate a prior distribution of operators and operands. This method solves the problem of GP code bloat effectively. Another way of combining the two is to replace GP's genetic operators with NN. Wittenberg et al. [216] used **denoising autoencoder long short-term memory networks** (**DAE-LSTMs**) as a probabilistic model to replace the standard recombination and mutation operators of GP.

5.5 Other NN Models

Beyond the specific SR methods discussed, researchers have integrated NNs with constraint rules to tackle SR problems. For example, Udrescu and Tegmark [196] decomposed complex SR tasks into simpler subproblems by using NNs to identify properties like multiplicative separability and shift symmetry, followed by symbolic search. Similarly, Cranmer et al. [45] proposed fitting symbolic components of a trained **graph neural network** (**GNN**) independently, leveraging strong inductive biases, and combining subexpressions to derive the final algebraic formula.

Additionally, some approaches leverage the knowledge acquired by NNs, such as storing learned information in memory to guide the search process, enhancing the adaptability of the model to new tasks [53]. Further, Kusner et al. [116] developed a grammar variable autoencoder that can generate parse trees representing discrete objects using context-free syntax. Not only that, there are efforts to explicitly combine assumptions about the underlying true expression structure to perform SR, allowing the model to more specifically control the predicted expression structure [58, 129]. For example, Bendinelli et al. [18] proposed NSRwH, a NN-SR method that integrates structural assumptions into predictions, improving accuracy and enabling control over expression structure.

Kolmogorov-Arnold Networks (**KANs**) [132] have recently gained attention for their interpretability and structured approach to approximating multivariate functions. KANs replace fixed activation functions with learnable spline functions, eliminating reliance on linear weight matrices and enhancing adaptability to high-dimensional data. Many studies have enhanced KANs by improving spline flexibility for better function approximation [221] and adding regularization methods to boost generalization [61].

5.6 Summary

Table 4 provides a brief overview of representative NN-based methods and includes code links for further exploration. While DRL and Transformer models can significantly improve learning efficiency, their strong dependence on training data features often limits generalization, making them less adaptable to unseen data. EQL offers some interpretability through symbolic operations,

Table 4. NN-Based Methods For SR

Method	Description	Year	Code
Deep Reinforcement Learning			
DSR [170]	Risk-Seeking Policy Gradient	2019	App. 10
NGPPS [153]	RL,GP	2021	App. 10
RBG2-SR [47]	Domain-knowledge, Grammar guided	2022	App. 10
IRL-SR [46]	Human-Preference Feedback	2022	App. 10
Φ-SO [192]	Physical Symbolic Optimization, Units Constraints	2023	App. 10
Sym-Q [194]	Q-network, Offline RL	2024	App. 10
Transformer			
Sy-Mathematics [121]	Three Approaches To Generate Functions	2019	App. 10
SymbolicGPT [201]	Large Scale Pre-training	2021	App. 10
NeSymReS [20]	Language Model	2021	App. 10
E2E [82]	Symbolic-Numeric Vocabulary	2022	App. 10
JSL-SR [130]	Joint Supervised Learning	2023	App. 10
ODEforme [51]	Dynamical Systems, Ordinary Differential Equation	2023	App. 10
DGSR [75]	Pre-Trained Deep Generative	2024	App. 10
TPSR [187]	Monte Carlo Tree Search, Non-differentiable feedback	2023	App. 10
EQL			
EQL [140]	Equation Learner	2016	App. 10
EQL$^{\div}$ [179]	Equation Learner Division	2018	App. 10
IEQL [95]	Deep Learning Architectures	2020	App. 10
GMEQL [29]	Gumbel-Max Trick	2020	App. 10
FEQL [131]	Density Functional Theory	2020	App. 10
OccamNet [42]	Skip Connections, Evolutionary Strategy	2020	App. 10
MathONet [240]	DenseNet-Like Hierarchical Structure	2022	App. 10
Parametric-EQL [229]	CNN, Parametric Equations	2023	App. 10
DeepSymNet [219]	Two Smaller Subproblems	2023	App. 10
PruneSymNet [218]	Greedy Pruning, Beam Search	2024	App. 10
SymbolNet [195]	End-to-end single-phase Dynamic Pruning	2024	App. 10
Others			
GrammarVAE [116]	Grammar Variational Autoencoder	2017	App. 10
GNSR [45]	Particle Systems and Graph Networks	2020	App. 10
AI-Feynman [196]	NN Fitting With Physics-Inspired Techniques	2020	App. 10
EQDiscovery [38]	Physics Embedding, Sparse Regression	2021	App. 10
NSRwH [18]	User-Defined Prior Knowledge or Hypotheses	2023	App. 10
SISR [58]	Multi-Layer LSTM-RNN	2023	App. 10
KAN [132]	Kolmogorov-Arnold representation theorem	2024	App. 10

"NGPPS" indicts Neural-guided genetic programming population seeding; "RBG2-SR" indicts Reinforcement Based Grammar Guided Symbolic Regression; "IRL-SR" indicts Interactive Reinforcement Learning for Symbolic Regression; "Φ-SO" indicts Physical Symbolic Optimization; "Sym-Q" indicts Symbolic Qnetwork; "NeSymReS" indicts Neural Symbolic Regression That Scales; "E2E" indicts End-to-End; "TPSR" indicts Transformer-based Planning for Symbolic Regression; "NSRwH" indicts Neural Symbolic Regression with Hypotheses;"SISR" indicts Symplectically Integrated Symbolic Regression;

Table 5. State-of-the-art SR Methods

Name	Description	Year	code
Operon [24]	C++ coded GP, fine-tuning by Levenberg-Marquardt	2020	App. 10
SBP-GP [205]	Semantic back-propagation genetic programming	2019	App. 10
uDSR [122]	Neural-guided search, large-scale pre-training,	2022	App. 10
Bingo [175]	Acyclic graphs, Linear representation, coefficient tuning	2022	App. 10
E2ET [82]	Pre-trained transformer, fine-tuned	2022	App. 10
RILS-ROLS [87]	Local search and ordinary least squares	2023	App. 10
MRGP [10]	Multiple Regression GP	2014	App. 10
GP-GOMEA [206]	Gene-pool OptimalMixing	2021	App. 10
FEAT [118]	Feature Engineering Automation Tool	2019	App. 10
AIFeynman [196]	Physics-inspired SR	2020	App. 10

yet most NN approaches still suffer from the "black-box" nature, hindering their applicability in scenarios requiring transparency. Moreover, models like Transformers and DRL typically demand considerable computational resources and time, especially with high-dimensional data. Although they improve accuracy and efficiency, the computational cost remains a challenge for large-scale or complex tasks. Future research may focus on three key areas: enhancing generalization, improving interpretability, and reducing computational complexity. Generalization can be improved through adaptive regularization techniques (e.g., dropout, early stopping) and increased data diversity. Interpretability may benefit from integrating SR-generated symbolic expressions into the learning process. To reduce computational costs, optimizing NN architectures and training strategies such as pruning or knowledge distillation can help maintain performance while lowering resource demands.

6 State-of-the-art

Current SR algorithms exhibit varying performance and characteristics across different tasks. To systematically evaluate their practical effectiveness, we conducted a comprehensive review based on multiple SRBench benchmarks. We selected 10 representative algorithms with strong performance and promising potential, using criteria such as fitting accuracy, extrapolation ability, and formula simplicity. Table 5 summarizes key information about these algorithms, including their original publications, implementation availability, and main features. This provides researchers with a clear overview of the SR landscape and supports practitioners in selecting appropriate methods for specific applications.

Operon [24] is a high-performance SR framework combining evolutionary algorithms with efficient representations and optimization techniques. It emphasizes scalability and includes features like hybrid optimization, specialized mutation operators, and multi-threaded execution, making it ideal for large-scale, complex datasets. **SBP-GP** [205] enhances GP by incorporating semantic back-propagation, using fitness gradient information to guide evolutionary operations. This improves convergence speed and reduces premature convergence by ensuring semantically meaningful solutions. **uDSR** [122] uses deep reinforcement learning to search for symbolic expressions, employing NN to handle high-dimensional datasets effectively. By integrating domain-specific knowledge, uDSR achieves high accuracy and efficiency. **Bingo** [175] is a Python-based SR library focusing on flexibility and extensibility. It uses evolutionary algorithms with explicit control over function sets and constraints, making it effective for customized symbolic models. **RILS-ROLS** [87] combines iterative local search with regularized least squares fitting to optimize symbolic expressions. It balances global exploration and local refinement, excelling in problems with complex search

spaces. **E2ET** [82] uses deep learning for end-to-end SR, excelling in modeling highly complex functions, particularly with large datasets and high-dimensional problems. **MRGP** [10] combines multiple representation forms to enhance adaptability, ideal for tasks requiring flexible problem representation and optimization. **GP-GOMEA** [206] improves evolutionary processes with efficient gene-pool mixing, excelling in large-scale and computationally intensive tasks. **FEAT** [118] integrates SR with feature engineering to create interpretable models, especially useful in finance and healthcare. **AI Feynman** [196] uncovers physical formulas from data, making it particularly suited for scientific research in physics and engineering.

7 Applications Of SR

Over recent years, there has been growing interest among researchers in interpretable models, leading to significant traction for SR. It's noteworthy that the application scope of SR is extensive and rapidly expanding. Therefore, it's impractical to list all application scenarios in this article. This section will focus on five key application areas of SR.

7.1 Model Interpretation

SR offers a robust approach to interpreting complex models by generating explicit mathematical expressions that are human-readable and insightful. As a surrogate model, SR can approximate the input-output relationships of NN, replacing their opaque structures with interpretable formulas. For instance, Wetzel et al. [213] utilized SR to find equivalences between NN outputs and human-readable equations, providing an interpretable approximation of network behavior. SR also excels in local interpretability by generating formulas that explain model behavior in specific input regions, as Luo et al. [135] extracted semantic information from NN layers. Furthermore, SR can reveal hidden feature interactions within models, like Cava et al. [120] applied SR to high-dimensional electronic health record data, automating feature engineering and producing concise, accurate models. Beyond feature interactions, SR generates rule-based formulas that uncover embedded regularities, such as in the work of Zheng et al. [232], where NN-based **learning-to-optimize** (**L2O**) predictors were converted into symbolic representations with enhanced interpretability metrics. Nadizar et al. [155] utilized the generative process of GP to guide user preferences in online training of artificial NN, thereby enhancing interpretability.

7.2 Physical Modeling

Physical modeling is one of the most prevalent applications of SR, extensively used in dynamic systems modeling, materials science, climate and environmental science, and astrophysics. SR can derive equations describing various physical phenomena, such as motion, electromagnetic fields, and thermodynamics. For example, Tenachi et al. [192] proposed a physical symbolic optimization framework using deep reinforcement learning to recover analytical symbolic expressions from physical data through unit constraint learning. Medina and White [145] introduced active learning and physical constraints regularization into SR to enhance performance. Additionally, Angelisis et al. [8] provided detailed application scenarios in the physical domain. Key studies, such as Schmidt and Lipson's [183] work on automatically generating physical laws, and Udrescu and Tegmark's AI Feynman method [197], which combines physical knowledge to improve complex physical system modeling, highlight SR's potential. Cranmer et al. [45] discussed integrating deep learning with SR, while Sahoo et al. [179] explored SR's application in extrapolation and control. These studies demonstrate that SR, by generating interpretable mathematical models, can significantly improve our understanding and prediction of complex physical systems' behavior.

7.3 Materials Science

There are many potential areas in materials science where SR can be applied, primarily due to its unique capability in modeling nonlinear systems. Nonlinear phenomena are ubiquitous in materials science, especially in the changes in material properties induced by variations in structure, composition, and external disturbances [210]. For example, SR has been used to identify the relationship between the adsorption energy of the first zinc atom and the properties of dopant atoms [86]. Additionally, the discovery of material models is a key area of research with broad academic and technical significance. For instance, SR has been successfully applied to generate constitutive models for hyperelastic materials automatically [1]. In such contexts, dynamic multivariable models are ideal as they can uncover the complex interdependencies between variables and provide data-driven guidance for optimizing material performance. Through SR models, researchers can effectively identify the relationship between the microscopic properties and macroscopic performance of materials, thereby enabling quantitative predictions and improvements in material design [48].

7.4 Engineering Design

SR has been widely used in engineering design, it can help engineers find and optimize the design parameters of complex systems, to improve the design efficiency and effect. SR plays a crucial role in the fields of structural optimization, control system design, material design, and circuit design. For example, Kronberger et al. [110] applied SR to friction systems to determine optimal and safe design parameters; Danai et al. [50] use SR in controller design to improve interpretability. In addition, SR is also used to model and control robotic arms [231], as well as to model and control manufacturing systems [26, 54]. Going one step further, SR provides a powerful tool for electronic circuit design, not only to derive mathematical models of circuits but also to optimize circuit performance and analyze the effects of individual components [28, 182]. These studies show that SR significantly improves the interpretability and optimization ability of engineering design models.

7.5 Knowledge Discovery

SR is a powerful tool for knowledge discovery, providing insights into complex relationships in data by generating interpretable mathematical expressions that help researchers extract underlying patterns and relationships from complex data. For example, SR has been applied to simulate the transmission spectra of exoplanets during transits, allowing researchers to gain insight into the atmospheric structure and composition of these objects [141]. In the reference [106, 111], it describes how to use SR for knowledge discovery using the open source software HeuristicLab. The SR can also be extracted into a crowd behavior rules to help in large crowd control place [127, 233]. These applications show that SR significantly facilitates the ability to extract knowledge from a variety of data sets by generating insightful models.

8 Software Tools

With the increasing popularity of SR, there has been a notable surge in the development of software tools dedicated to applications such as data modeling, predictive analysis, and system understanding. This subsection aims at providing an overview and analysis of some tools, as detailed in Table 6.

Cava et al. [117] developed **SRBench**, an open-source platform evaluating 14 SR and 7 machine learning methods on 252 regression problems, including real-world datasets and benchmark problems like physical equations and ODE systems. It serves as a comprehensive tool for SR beginners. Extended as **SRBench++** [52], it provides detailed comparisons of 12 SR algorithms. Additionally, Python packages like **CP3-bench** and Aldeia et al.'s [6] library for benchmarking interpretability

Table 6. Software Tools For SR

Tools	Description	Type	Implementation
SRBench [117]	Integrated platform	GP,NN,DE	Python,C++,C[10]
SRBench++ [52]	Integrated platform	GP,NN,DE	Python,C++,Java[10]
CP3-bench [193]	Integrated platform	GP,NN,DE	Python,C[10]
iirsBenchmark [6]	Explanatory platform	VTRM	Python,jupyter[10]
GPLab [214]	A GP toolbox for MATLAB	GP	Matlab[10]
GenProg [124]	Automatic software repair	GP	C[10]
GPC++	Automatically defined functions	GP	C++[10]
ellenGP	Stack-based, linear representation	GP	C++[10]
GPTIPS [186]	Interactive modelling environment	GP	MATLAB[10]
PonyGE2 [66]	Formal Grammars	GE	Python[10]
WebGE [41]	Web-based graphical user interface	GE	Java[10]
Eureqa [183]	User-friendly GUI	GP	C++[10]
QLattice [22]	Quantum inspiration	–	Python[10]
Glyph [173]	A lightweight framework	GP	Python[10]
DEAP [44]	Evolutionary Computation Framework	EC	Python[10]
GPlearn [189]	A Scikit-Learn compatible API	GP	Python[10]
PySR [44]	Multi-population, Asynchronous	GP	Python, Julia[10]
HeuristicLab [63]	Heuristic and evolutionary algorithms	GP	C #[10]
Data-Modeler [100]	Evolved Analytics' DataModeler	GP	–[10]
ALAMO [43]	Machine learning software	GP	Python,C[10]
HyperGP	Heterogeneous Parallel	GP	Python,C++,Cuda[10]

"GP" indicates genetic programming; "NN" indicates neural network; "DE" indicates deterministic methods; "GE" indicates grammatical evolution; "VTRM"indicates various types of regression methods. The "Implementation" column lists some commonly used languages; further details can be accessed through the link.

methods have further enriched SR benchmarking resources. **GPLab** [214] and **GPTIPS** [186] are two software toolboxes designed based on MATLAB, facilitating the implementation of GP and SR techniques. They support the exploration of complex mathematical relationships within data through evolutionary algorithms. **PonyGE2** [66] and **WebGE** [41] are two software tools based on **grammatical evolution** (**GE**). PonyGE2 is a Python framework offering a flexible environment for evolutionary computation research, while WebGE is a web-based platform providing an accessible interface for applying GE techniques through a browser. **Eureqa** [183] is a commercial SR software that is user-friendly with a simple and intuitive **graphical user interface** (**GUI**). It also allows operations through a **command-line interface** (**CLI**), providing users with flexibility in their interactions with the software. **QLattice** [22] is an SR tool developed by Abzu and Aibrain. QLattice supports users in setting up SR tasks through simple drag-and-drop and interactive methods. It possesses adaptability, automatically selecting and adjusting models to better suit various types of data. **HeuristicLab** [63] is an open-source software platform for modeling and optimizing problems. HeuristicLab is a cross-platform software that can run on operating systems such as Windows, Linux, and macOS. **Data-Modeler** [100] is a software tool for creating and optimizing mathematical models based on collected data. **Automated Learning and Modeling Optimization (ALAMO)** [43] is a machine learning software designed to construct accurate and interpretable models using data collected from experiments or other black-box systems. **HyperGP** is an open-source high performance framework, providing convenient distributed heterogeneous acceleration for the custom prototyping of GP and its variants.

Furthermore, the implementation of SR can be accomplished through various tools and libraries, with specific choices depending on the user's needs, preferences, and use cases. In the Python ecosystem, libraries such as **DEAP** [68], **GPlearn** [189], Glyph [173], **PySR** [44], and **Pyevolve** [169] offer convenient functionalities for SR. Additionally, there are other toolkits like **Evolutionary Computation in Java (ECJ)** [185] that provide corresponding implementations on the Java platform. These tools and libraries furnish the fundamental framework and algorithms for SR, allowing users to customize and extend them based on the specific characteristics of their problems.

9 Future Trends

9.1 Interpretable Models

SR has been the focus of extensive efforts to enhance interpretability from various perspectives, including model transparency, usability, and simplicity. Some studies have also incorporated human preferences as prior knowledge to develop more interpretable SR models [155]. While these approaches have made notable progress, future research may focus on: (1) further improving the generalization ability of SR models to maintain interpretability on more complex tasks; and (2) enhancing usability by making the derivation process more intuitive and accessible, thereby lowering cognitive and practical barriers for human users.

With the growing demand for interpretability in black-box models such as NNs, explaining **machine learning** (**ML**) models has become a major focus in both academia and industry [72]. SR, which reveals underlying data relationships through explicit mathematical expressions, offers unique advantages for enhancing interpretability. However, its use in explaining NNs remains relatively underexplored, leaving ample room for future research. Promising directions include: (1) integrating SR with NN architectures to derive intuitive mathematical representations of network behavior; (2) applying SR to large-scale, complex NNs to achieve a balance between model compression and interpretability.

9.2 SR Meets Deep Learning

To further harness the strengths of both deep learning and SR, future research may focus on embedding symbolic reasoning into deep learning frameworks. This includes integrating techniques such as logical inference and rule-based reasoning into NNs, enabling them to preserve hierarchical symbolic structures and reasoning capabilities during learning. Such integration aims at combining deep learning's capacity for handling large-scale data with SR's interpretability and reasoning strengths. Conversely, deep learning can also enhance SR by assisting in data preprocessing, feature extraction, and refinement of the SR process itself. For instance, **convolutional neural networks** (**CNNs**) and **recurrent neural networks** (**RNNs**) can automatically extract informative features that serve as inputs to SR, improving its understanding of complex data patterns. Furthermore, combining SR with large language models (LLMs)—which excel at capturing intricate patterns and associations—holds promise for generating more accurate and reliable mathematical expressions, thereby advancing automated scientific discovery.

9.3 SR Meets AutoML

Embedding SR into the AutoML process brings comprehensive and flexible advantages. This approach not only automates the design of machine learning pipelines, including data preprocessing, feature engineering, and model selection but also enhances model performance by optimizing model structure and searching the hyperparameter space. The flexibility of SR enables the AutoML system to adapt to various tasks and data types, providing users with customized solutions. Furthermore, AutoML can be employed for feature engineering and model selection, facilitating

the automatic extraction of data features and the identification of the most suitable SR model to enhance model performance and generalization capability. These AutoML algorithms proficiently extract pertinent features from raw data and identify optimal model structures and parameter configurations using optimization algorithms. As a result, they minimize the requirement for manual intervention and elevate the accuracy and efficiency of SR models.

9.4 SR Meets Multi-form Equations

SR faces challenges in modeling dynamic systems like ODEs, with time dependencies and nonlinear relationships. Future research could address these by incorporating time-series data, using deep learning for nonlinear systems, and applying multi-scale modeling for dynamic features. SR also struggles with implicit equations, which are difficult to express and require solving complex systems, increasing computational complexity. Future work should focus on designing suitable loss functions, improving interpretability, and combining deep learning with SR for better solution accuracy and efficiency.

9.5 Exploration of Multimodal SR

Multimodal SR is an emerging research trend that aims at handling diverse data types (e.g., images, text, sound) and extract symbolic expressions for more comprehensive interpretation and prediction of complex phenomena. The focus is on integrating information from different modalities to improve SR accuracy and robustness. However, multimodal SR faces several challenges, including the complexity of data fusion, model design, and task definition. Effectively integrating information from different modalities, designing SR models suitable for multimodal data, and defining clear task objectives are key issues that need to be addressed.

9.6 Open Source Platform

A key future direction in SR is the development of open-source universal platforms. While benchmarks like SRBench [117] and SRBench++ [52] are useful, they lack dataset diversity, particularly in high-dimensional, noisy, or imbalanced scenarios. Evaluation metrics are also limited and may not capture all performance differences. Future improvements could include expanding datasets, introducing more complex problems (e.g., implicit functions), developing comprehensive metrics, and creating dynamic benchmarking frameworks. Another key advancement will be the development of flexible, easy-to-use, and parallelizable open-source GP platforms.

10 Conclusions

With the development of explainable AI, various methods enhancing model interpretability have been proposed in the research community. Compared to traditional black-box models, SR demonstrates significant competitiveness due to its capability to derive explicit and interpretable mathematical expressions. Therefore, SR has become a hot research topic.

In this survey, we first introduce the background information of SR to help readers better understand it. Secondly, an overview of the latest SR technologies is provided, mainly categorized into three aspects: deterministic methods, GP-based methods, and NN-based methods. We then selected and introduced ten advanced methods, providing links to their implementations. Subsequently, five key applications are summarized to show the potential ability of SR to handle real-world problems. We also summarize and list the commonly used toolkits and software.

Despite the rapid progress and development of SR, there are still many challenges and unresolved issues. Firstly, there is a need to further enhance the interpretability of models, including improving the interpretability of SR models themselves and applying SR to interpret other models. Secondly, integrating SR with deep learning, and applying symbolic reasoning to deep learning

models, is an important direction. Thirdly, exploring the combination of SR and automatic learning is also a promising research direction. Fourthly, advancing multi-form symbolic structures for SR and multimodel SR to address complex real-world problems is crucial. Lastly, creating open-source universal platforms is critical for advancing the field.

References

[1] Rasul Abdusalamov, Markus Hillgärtner, and Mikhail Itskov. 2023. Hyperelastic material modelling using symbolic regression. *PAMM* 22, 1 (2023), e202200263.

[2] Baligh Al-Helali, Qi Chen, Bing Xue, and Mengjie Zhang. 2021. Multitree genetic programming with new operators for transfer learning in symbolic regression with incomplete data. *IEEE Transactions on Evolutionary Computation* 25, 6 (2021), 1049–1063.

[3] Baligh Al-Helali, Qi Chen, Bing Xue, and Mengjie Zhang. 2024. Genetic programming for feature selection based on feature removal impact in high-dimensional symbolic regression. *IEEE Transactions on Emerging Topics in Computational Intelligence* 8, 3 (2024), 2269–2282.

[4] Baligh Al-Helali, Qi Chen, Bing Xue, and Mengjie Zhang. 2024. Multitree genetic programming with feature-based transfer learning for symbolic regression on incomplete data. *IEEE Transactions on Cybernetics* 54, 7 (2024), 4041–4027.

[5] Guilherme Seidyo Imai Aldeia and Fabrício Olivetti de França. 2021. Measuring feature importance of symbolic regression models using partial effects. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 750–758.

[6] Guilherme Seidyo Imai Aldeia and Fabricio Olivetti de Franca. 2022. Interpretability in symbolic regression: A benchmark of explanatory methods using the feynman data set. *Genetic Programming and Evolvable Machines* 23, 3 (2022), 309–349.

[7] Muhammad Sarmad Ali, Meghana Kshirsagar, Enrique Naredo, and Conor Ryan. 2021. Towards automatic grammatical evolution for real-world symbolic regression. In *Proceedings of the IJCCI*. 68–78.

[8] Dimitrios Angelis, Filippos Sofos, and Theodoros E. Karakasidis. 2023. Artificial intelligence in physical sciences: Symbolic regression trends and perspectives. *Archives of Computational Methods in Engineering* 30, 6 (2023), 3845–3865.

[9] Aftab Anjum, Fengyang Sun, Lin Wang, and Jeff Orchard. 2019. A novel neural network-based symbolic regression method: Neuro-encoded expression programming. In *Proceedings of the International Conference on Artificial Neural Networks* 373–386.

[10] Ignacio Arnaldo, Krzysztof Krawiec, and Una-May O'Reilly. 2014. Multiple regression genetic programming. In *Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation*. 879–886.

[11] Arthur Asuncion and David Newman. 2007. UCI machine learning repository. https://ergodicity.net/2013/07/

[12] Douglas A. Augusto and Helio JC Barbosa. 2013. Accelerated parallel genetic programming tree evaluation with OpenCL. *Journal of Parallel and Distributed Computing* 73, 1 (2013), 86–100.

[13] Vernon Austel, Cristina Cornelio, Sanjeeb Dash, Joao Goncalves, Lior Horesh, Tyler Josephson, and Nimrod Megiddo. 2020. Symbolic regression using mixed-integer nonlinear optimization. arXiv:2006.06813. Retrieved from https://arxiv.org/abs/2006.06813

[14] Vernon Austel, Sanjeeb Dash, Oktay Gunluk, Lior Horesh, Leo Liberti, Giacomo Nannicini, and Baruch Schieber. 2017. Globally optimal symbolic regression. https://arxiv.org/abs/1710.10720

[15] Irene Azzali, Leonardo Vanneschi, Sara Silva, Illya Bakurov, and Mario Giacobini. 2019. A vectorial approach to genetic programming. In *Proceedings of the European Conference on Genetic Programming*. 213–227.

[16] Deaglan J. Bartlett, Harry Desmond, and Pedro G. Ferreira. 2024. Exhaustive symbolic regression. *IEEE Transactions on Evolutionary Computation* 28, 4 (2024), 950–964.

[17] Lawrence Beadle and Colin G. Johnson. 2008. Semantically driven crossover in genetic programming. In *Proceedings of the 2008 IEEE Congress on Evolutionary Computation*. 111–116.

[18] Tommaso Bendinelli, Luca Biggio, and Pierre-Alexandre Kamienny. 2023. Controllable neural symbolic regression. In *Proceedings of the International Conference on Machine Learning*. 2063–2077.

[19] Luca Biggio, Tommaso Bendinelli, Aurelien Lucchi, and Giambattista Parascandolo. 2020. A seq2seq approach to symbolic regression. In *Proceedings of the Advances in Neural Information Processing Systems*.

[20] Luca Biggio, Tommaso Bendinelli, Alexander Neitz, Aurelien Lucchi, and Giambattista Parascandolo. 2021. Neural symbolic regression that scales. In *Proceedings of the Advances in Neural Information Processing Systems*. 936–945.

[21] Luca Biggio, Bendinelli Tommaso, and Pierre-Alexandre Kamienny. 2022. Privileged deep symbolic regression. In *Proceedings of the International Conference on Machine Learning*.

[22] Kevin René Broløs, Meera Vieira Machado, Chris Cave, Jaan Kasak, Valdemar Stentoft-Hansen, Victor Galindo Batanero, Tom Jelen, and Casper Wilstrup. 2021. An approach to symbolic regression using feyn. arXiv:2104.05417. Retrieved from https://arxiv.org/abs/2104.05417

[23] Karina Brotto Rebuli, Mario Giacobini, Sara Silva, and Leonardo Vanneschi. 2023. A comparison of structural complexity metrics for explainable genetic programming. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 539–542.

[24] Bogdan Burlacu, Gabriel Kronberger, and Michael Kommenda. 2020. Operon C++ an efficient genetic programming framework for symbolic regression. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 1562–1570.

[25] Bogdan Burlacu, Kaifeng Yang, and Michael Affenzeller. 2024. Population diversity and inheritance in genetic programming for symbolic regression. *Natural Computing* 23, 3 (2024), 531–566.

[26] Birkan Can and Cathal Heavey. 2011. Comparison of experimental designs for simulation-based symbolic regression of manufacturing systems. *Computers and Industrial Engineering* 61, 3 (2011), 447–462.

[27] Lulu Cao, Zimo Zheng, Chenwen Ding, Jinkai Cai, and Min Jiang. 2023. Genetic programming symbolic regression with simplification-pruning operator for solving differential equations. In *Proceedings of the Advances in Neural Information Processing Systems*. 287–298.

[28] Vladimir Ceperic, Niko Bako, and Adrijan Baric. 2014. A symbolic regression-based modelling strategy of AC/DC rectifiers for RFID applications. *Expert Systems with Applications* 41, 16 (2014), 7061–7067.

[29] Gang Chen. 2020. Learning symbolic expressions via gumbel-max equation learner networks. arXiv:2012.06921. Retrieved from https://arxiv.org/abs/2012.06921

[30] Qi Chen, Bing Xue, Yi Mei, and Mengjie Zhang. 2017. Geometric semantic crossover with an angle-aware mating scheme in genetic programming for symbolic regression. In *Genetic Programming: 20th European Conference, EuroGP 2017, Amsterdam, The Netherlands, April 19-21, 2017, Proceedings 20*. 229–245.

[31] Qi Chen, Bing Xue, Yi Mei, and Mengjie Zhang. 2017. Geometric semantic crossover with an angle-aware mating scheme in genetic programming for symbolic regression. In *Genetic Programming: 20th European Conference, EuroGP 2017, Amsterdam, The Netherlands, April 19-21, 2017, Proceedings 20*. 229–245.

[32] Qi Chen, Bing Xue, Ben Niu, and Mengjie Zhang. 2016. Improving generalisation of genetic programming for high-dimensional symbolic regression with feature selection. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 3793–3800.

[33] Qi Chen, Bing Xue, and Mengjie Zhang. 2018. Improving generalization of genetic programming for symbolic regression with angle-driven geometric semantic operators. *IEEE Transactions on Evolutionary Computation* 23, 3 (2018), 488–502.

[34] Qi Chen, Bing Xue, and Mengjie Zhang. 2020. Genetic programming for instance transfer learning in symbolic regression. *IEEE Transactions on Cybernetics* 52, 1 (2020), 25–38.

[35] Qi Chen, Bing Xue, and Mengjie Zhang. 2020. Preserving population diversity based on transformed semantics in genetic programming for symbolic regression. *IEEE Transactions on Evolutionary Computation* 25, 3 (2020), 433–447.

[36] Qi Chen, Mengjie Zhang, and Bing Xue. 2017. Feature selection to improve generalization of genetic programming for high-dimensional symbolic regression. *IEEE Transactions on Evolutionary Computation* 21, 5 (2017), 792–806.

[37] Yongliang Chen, Jinghui Zhong, and Mingkui Tan. 2018. Comprehensive learning gene expression programming for automatic implicit equation discovery. In *Proceedings of the Computational Science*. 114–128.

[38] Zhao Chen, Yang Liu, and Hao Sun. 2021. Physics-informed learning of governing equations from scarce data. *Nature Communications* 12, 1 (2021), 6136.

[39] Thi Huong Chu, Quang Uy Nguyen, and Van Loi Cao. 2018. Semantics based substituting technique for reducing code bloat in genetic programming. In *Proceedings of the International Symposium on Communications and Information Technologies*. 77–83.

[40] Thi Huong Chu, Quang Uy Nguyen, and Michael O'Neill. 2018. Semantic tournament selection for genetic programming based on statistical analysis of error vectors. *Information Sciences* 436 (2018), 352–366.

[41] J. Manuel Colmenar, Raúl Martín-Santamaría, and J. Ignacio Hidalgo. 2022. WebGE: An open-source tool for symbolic regression using grammatical evolution. In *Proceedings of the International Conference on the Applications of Evolutionary Computation*. 269–282.

[42] Allan Costa, Rumen Dangovski, Owen Dugan, Samuel Kim, Pawan Goyal, Marin Soljačić, and Joseph Jacobson. 2020. Fast neural models for symbolic regression at scale. arXiv:2007.10784. Retrieved from https://arxiv.org/abs/2007.10784

[43] Alison Cozad and Nikolaos V. Sahinidis. 2018. A global MINLP approach to symbolic regression. *Mathematical Programming* 170 (2018), 97–119.

[44] Miles Cranmer. 2023. Interpretable machine learning for science with PySR and SymbolicRegression.jl. arXiv:2305.01582. Retrieved from https://arxiv.org/abs/2305.01582

[45] Miles Cranmer, Alvaro Sanchez Gonzalez, Peter Battaglia, Rui Xu, Kyle Cranmer, David Spergel, and Shirley Ho. 2020. Discovering symbolic models from deep learning with inductive biases. *In Proceedings of the Advances in Neural Information Processing Systems* 33 (2020), 17429–17442.
[46] Laure Crochepierre, Lydia Boudjeloud-Assala, and Vincent Barbesant. 2022. Interactive reinforcement learning for symbolic regression from multi-format human-preference feedbacks. In *Proceedings of the Int. Joint Conf. Artif. Intell*
[47] Laure Crochepierre, Lydia Boudjeloud-Assala, and Vincent Barbesant. 2022. A reinforcement learning approach to domain-knowledge inclusion using grammar guided symbolic regression. arXiv:2202.04367. Retrieved from https://arxiv.org/abs/2202.04367
[48] Daniel Cunnington, Flaviu Cipcigan, Rodrigo Neumann Barros Ferreira, and Jonathan Booth. 2023. Symbolic learning for material discovery. arXiv:2312.11487. Retrieved from https://arxiv.org/abs/2312.11487
[49] Léo Françoso dal Piccol Sotto and Vinícius Veloso de Melo. 2016. Studying bloat control and maintenance of effective code in linear genetic programming for symbolic regression. *Neurocomputing* 180 (2016), 79–93.
[50] Kourosh Danai and William G. La Cava. 2021. Controller design by symbolic regression. *Mechanical Systems and Signal Processing* 151 (2021), 107348.
[51] Stéphane d'Ascoli, Sören Becker, Alexander Mathis, Philippe Schwaller, and Niki Kilbertus. 2023. ODEFormer: Symbolic regression of dynamical systems with transformers. *In Proceedings of the International Conference on Learning Representations* (2023). https://doi.org/10.48550/arXiv.2310.05573
[52] FO de Franca, M. Virgolin, M. Kommenda, MS Majumder, M. Cranmer, G. Espada, L. Ingelse, A. Fonseca, M. Landajuela, B. Petersen, et al. 2024. SRBench++: Principled benchmarking of symbolic regression with domain-expert interpretation. *IEEE Transactions on Evolutionary Computation* (2024).
[53] AK Deklel, AM Hamdy, and Elsayed M. Saad. 2018. Multi-objective symbolic regression using long-term artificial neural network memory (LTANN-MEM) and neural symbolization algorithm (NSA). *Neural Computing and Applications* 29 (2018), 935–942.
[54] Erik Derner, Jiří Kubalík, Nicola Ancona, and Robert Babuška. 2020. Constructing parsimonious analytic models for dynamic systems via symbolic regression. *Applied Soft Computing* 94 (2020), 106432.
[55] Grant Dick, Caitlin A Owen, and Peter A Whigham. 2020. Feature standardisation and coefficient optimisation for effective symbolic regression. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 306–314.
[56] Li Ding, Ryan Boldi, Thomas Helmuth, and Lee Spector. 2022. Lexicase selection at scale. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 2054–2062.
[57] Li Ding, Edward Pantridge, and Lee Spector. 2023. Probabilistic lexicase selection. *In Proceedings of the Genetic and Evolutionary Computation Conference* (2023), 1073–1081.
[58] Daniel M. DiPietro and Bo Zhu. 2022. Symplectically integrated symbolic regression of hamiltonian dynamical systems. arXiv:1409.0473. Retrieved from https://arxiv.org/abs/1701.00133
[59] Junlan Dong, Jinghui Zhong, Wei-Neng Chen, and Jun Zhang. 2023. An efficient federated genetic programming framework for symbolic regression. *IEEE Transactions on Emerging Topics in Computational Intelligence* 7, 3 (2023), 858–871.
[60] Junlan Dong, Jinghui Zhong, Wei-Li Liu, and Jun Zhang. 2024. Evolving equation learner for symbolic regression. *IEEE Transactions on Evolutionary Computation* (2024).
[61] Ivan Drokin. 2024. Kolmogorov-arnold convolutions: Design principles and empirical studies. arXiv:2407.01092. Retrieved from https://arxiv.org/abs/2407.01092
[62] Stéphane d'Ascoli, Pierre-Alexandre Kamienny, Guillaume Lample, and Francois Charton. 2022. Deep symbolic regression for recurrence prediction. In *Proceedings of the International Conference on Machine Learning*. 4520–4536.
[63] Achiya Elyasaf and Moshe Sipper. 2014. Software review: The heuristiclab framework. *Genet. Program. Evolvable Mach.* 15 (2014), 215–218.
[64] Marissa R. Engle and Nikolaos V. Sahinidis. 2022. Deterministic symbolic regression with derivative information: General methodology and application to equations of state. *AIChE Journal* 68, 6 (2022), e17457.
[65] Jacques A. Esterhuizen, Bryan R. Goldsmith, and Suljo Linic. 2022. Interpretable machine learning for knowledge generation in heterogeneous catalysis. *Nature Catalysis* 5, 3 (2022), 175–184.
[66] Michael Fenton, James McDermott, David Fagan, Stefan Forstenlechner, Erik Hemberg, and Michael O'Neill. 2017. Ponyge2: Grammatical evolution in python. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 1194–1201.
[67] Francisco Fernández, Marco Tomassini, William F. Punch, and JM Sanchez. 2000. Experimental study of multipopulation parallel genetic programming. In *Proceedings of the Eur. Conf. Genetic Program.* 283–293.
[68] Félix-Antoine Fortin, François-Michel De Rainville, Marc-André Gardner Gardner, Marc Parizeau, and Christian Gagné. 2012. DEAP: Evolutionary algorithms made easy. *Journal of Machine Learning Research* 13, 1 (2012), 2171–2175.

[69] Lei Gan, Hao Wu, and Zheng Zhong. 2022. Integration of symbolic regression and domain knowledge for interpretable modeling of remaining fatigue life under multistep loading. *International Journal of Fatigue* 161 (2022), 106889.
[70] Nikola Gligorovski and Jinghui Zhong. 2023. LGP-VEC: A vectorial linear genetic programming for symbolic regression. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 579–582.
[71] Riccardo Guidotti. 2021. Evaluating local explanation methods on ground truth. *Artificial Intelligence* 291 (2021), 103428.
[72] Riccardo Guidotti, Anna Monreale, Salvatore Ruggieri, Franco Turini, Fosca Giannotti, and Dino Pedreschi. 2018. A survey of methods for explaining black box models. *ACM Computing Surveys* 51, 5 (2018), 1–42.
[73] Jiaming Guo, Rui Zhang, Shaohui Peng, Qi Yi, Xing Hu, Ruizhi Chen, Zidong Du, Ling Li, Qi Guo, Yunji Chen, et al. 2024. Efficient symbolic policy learning with differentiable symbolic expression. *In Proceedings of the Advances in Neural Information Processing Systems* 36 (2024).
[74] C. Haider, FO de Franca, B. Burlacu, and G. Kronberger. 2023. Shape-constrained multi-objective genetic programming for symbolic regression. *Applied Soft Computing* 132 (2023), 109855.
[75] Samuel Holt, Zhaozhi Qian, and Mihaela van der Schaar. 2022. Deep generative symbolic regression. In *Proceedings of the International Conference on Learning Representations*. 1–42. https://doi.org/10.48550/arXiv.2401.00282
[76] Zhixing Huang, Yi Mei, Fangfang Zhang, and Mengjie Zhang. 2024. Multitask linear genetic programming with shared individuals and its application to dynamic job shop scheduling. *IEEE Transactions on Evolutionary Computation* 28, 6 (2024), 1546–1560.
[77] Zhixing Huang, Yi Mei, and Jinghui Zhong. 2022. Semantic linear genetic programming for symbolic regression. *IEEE Trans. Cybern.* 54, 2 (2022), 1321–1334.
[78] Zhixing Huang, Jinghui Zhong, Liang Feng, Yi Mei, and Wentong Cai. 2020. A fast parallel genetic programming framework with adaptively weighted primitives for symbolic regression. *Soft Comput.* 24 (2020), 7523–7539.
[79] Noman Javed, Fernand Gobet, and Peter Lane. 2022. Simplification of genetic programs: A literature survey. *Data Mining and Knowledge Discovery* 36, 4 (2022), 1279–1300.
[80] Hoseong Jeong, Jae Hyun Kim, Seung-Ho Choi, Seokin Lee, Inwook Heo, and Kang Su Kim. 2022. Semantic cluster operator for symbolic regression and its applications. *Advances in Engineering Software* 172 (2022), 103174.
[81] Ying Jin, Weilin Fu, Jian Kang, Jiadong Guo, and Jian Guo. 2019. Bayesian symbolic regression. arXiv:1910.08892. Retrieved from https://arxiv.org/abs/1910.08892
[82] Pierre-Alexandre Kamienny, Stéphane d'Ascoli, Guillaume Lample, and François Charton. 2022. End-to-end symbolic regression with transformers. *In Proceedings of the Advances in Neural Information Processing Systems* 35 (2022), 10269–10281.
[83] Pierre-Alexandre Kamienny, Guillaume Lample, Sylvain Lamprier, and Marco Virgolin. 2023. Deep generative symbolic regression with monte-carlo-tree-search. In *Proceedings of the International Conference on Machine Learning*. 15655–15668.
[84] Lukas Kammerer, Gabriel Kronberger, Bogdan Burlacu, Stephan M. Winkler, Michael Kommenda, and Michael Affenzeller. 2020. Symbolic regression by exhaustive search: Reducing the search space using syntactical constraints and efficient semantic structure deduplication. *Genetic Programming Theory and Practice XVII* (2020), 79–99.
[85] Lukas Kammerer, Gabriel Kronberger, and Michael Kommenda. 2022. Symbolic regression with fast function extraction and nonlinear least squares optimization. In *Proceedings of the International Conference on Computer Aided Systems Theory*. 139–146.
[86] Mengde Kang, Kai Huang, Jiahui Li, Honglai Liu, and Cheng Lian. 2024. Theoretical design of dendrite-free zinc anode through intrinsic descriptors from symbolic regression. *Journal of Materials Informatics* 4, 2 (2024), N–A.
[87] Aleksandar Kartelj and Marko Djukanović. 2023. RILS-ROLS: Robust symbolic regression via iterated local search and ordinary least squares. *Journal of Big Data* 10, 71 (2023), 1–28.
[88] Ahmed Kattan and Yew-Soon Ong. 2015. Surrogate genetic programming: A semantic aware evolutionary search. *Information Sciences* 296 (2015), 345–359.
[89] Maarten Keijzer. 2003. Improving symbolic regression with interval arithmetic and linear scaling. In *Proceedings of the European Conference on Genetic Programming*. 70–82.
[90] Maarten Keijzer and Vladan Babovic. 1999. Dimensionally aware genetic programming. In *Proceedings of the 1st Annual Conference on Genetic and Evolutionary Computation*. 1069–1076.
[91] Johannes Kepler. 1953. *Epitome Astronomiae Copernicanae*. Jo. Plancus.
[92] Liron Simon Keren, Alex Liberzon, and Teddy Lazebnik. 2023. A computational framework for physics-informed symbolic regression with straightforward integration of domain knowledge. *Scientific Reports* 13, 1 (2023), 1249.
[93] Jongeun Kim, Sven Leyffer, and Prasanna Balaprakash. 2023. Learning symbolic expressions: Mixed-integer formulations, cuts, and heuristics. *Informs Journal on Computing* 35, 6 (2023), 1383–1403.

[94] Joanne T Kim, Sookyung Kim, and Brenden K. Petersen. 2021. An interactive visualization platform for deep symbolic regression. In *Proceedings of the 29th International Conference on International Joint Conferences on Artificial Intelligence*. 5261–5263.

[95] Samuel Kim, Peter Y. Lu, Srijon Mukherjee, Michael Gilbert, Li Jing, Vladimir Čeperić, and Marin Soljačić. 2020. Integration of neural network-based symbolic regression in deep learning for scientific discovery. *IEEE Transactions on Neural Networks and Learning Systems* 32, 9 (2020), 4166–4177.

[96] Kenneth E. Kinnear and Peter J. Angeline. 1994. *Advances in Genetic Programming*.

[97] Jon Klein and Lee Spector. 2007. Unwitting distributed genetic programming via asynchronous JavaScript and XML. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 1628–1635.

[98] Michael Kommenda, Gabriel Kronberger, Michael Affenzeller, Stephan M. Winkler, and Bogdan Burlacu. 2016. Evolving simple symbolic regression models by multi-objective genetic programming. *Genetic Programming Theory and Practice XIII* (2016), 1–19.

[99] Michael F. Korns. 2011. Accuracy in symbolic regression. *Genetic Programming Theory and Practice IX* (2011), 129–151.

[100] Mark E. Kotanchek, Ekaterina Vladislavleva, and Guido Smits. 2013. Symbolic regression is not enough: It takes a village to raise a model. *Genetic Programming Theory and Practice X* (2013), 187–203.

[101] John R. Koza. 1994. Genetic programming as a means for programming computers by natural selection. *Statistics and Computing* 4 (1994), 87–112.

[102] John R. Koza. 1994. *Genetic Programming II: Automatic Discovery of Reusable Programs*. MIT Press.

[103] John R. Koza et al. 1989. Hierarchical genetic algorithms operating on populations of computer programs. In *Proceedings of the Int. Joint Conf. Artif. Intell.* 768–774.

[104] Krzysztof Krawiec. 2012. Medial crossovers for genetic programming. In *Proceedings of the European Conference on Genetic Programming*. 61–72.

[105] Krzysztof Krawiec and Tomasz Pawlak. 2013. Approximating geometric crossover by semantic backpropagation. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 941–948.

[106] Gabriel Kronberger. 2011. Symbolic regression for knowledge discovery: Bloat, overfitting, and variable interaction networks. *ACM SIGEVOlution* 5, 4 (2011), 25–25.

[107] Gabriel Kronberger, J. Manuel Colmenar, Stephan M. Winkler, and J. Ignacio Hidalgo. 2020. Multilayer analysis of population diversity in grammatical evolution for symbolic regression. *Soft Comput.* 24 (2020), 11283–11295.

[108] Gabriel Kronberger, Fabricio Olivetti de França, Bogdan Burlacu, Christian Haider, and Michael Kommenda. 2022. Shape-constrained symbolic regression—improving extrapolation with prior knowledge. *Evolutionary Computation* 30, 1 (2022), 75–98.

[109] Gabriel Kronberger, Lukas Kammerer, Bogdan Burlacu, Stephan M. Winkler, Michael Kommenda, and Michael Affenzeller. 2019. Cluster analysis of a symbolic regression search space. *Genetic Programming Theory and Practice XVI* (2019), 85–102.

[110] Gabriel Kronberger, Michael Kommenda, Andreas Promberger, and Falk Nickel. 2018. Predicting friction system performance with symbolic regression and genetic programming with factor variables. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 1278–1285.

[111] Gabriel Kronberger, Stefan Wagner, Michael Kommenda, Andreas Beham, Andreas Scheibenpflug, and Michael Affenzeller. 2012. Knowledge discovery through symbolic regression with heuristiclab. In *Proceedings of the Machine Learning and Knowledge Discovery in Databases: European Conference*. 824–827.

[112] Jiří Kubalík, Erik Derner, and Robert Babuška. 2020. Symbolic regression driven by training data and prior knowledge. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 958–966.

[113] Jiří Kubalík, Erik Derner, and Robert Babuška. 2021. Multi-objective symbolic regression for physics-aware dynamic modeling. *Expert Systems with Applications* 182 (2021), 115210.

[114] Jiří Kubalík, Erik Derner, and Robert Babuška. 2023. Neural networks for symbolic regression. arXiv:2302.00773. Retrieved from https://arxiv.org/abs/2302.00773

[115] Jun-Ichi Kushida, Akira Hara, and Tetsuyuki Takahama. 2018. Cartesian genetic programming with module mutation for symbolic regression. In *Proceedings of the 2018 IEEE International Conference on Systems, Man, and Cybernetics*. 159–164.

[116] Matt J. Kusner, Brooks Paige, and José Miguel Hernández-Lobato. 2017. Grammar variational autoencoder. In *Proceedings of the International Conference on Machine Learning*. 1945–1954.

[117] William La Cava, Bogdan Burlacu, Marco Virgolin, Michael Kommenda, Patryk Orzechowski, Fabrício Olivetti de França, Ying Jin, and Jason H. Moore. 2021. Contemporary symbolic regression methods and their relative performance. *Adv. Neural Inf. Process Syst.* 2021, DB1 (2021), 1–16.

[118] William La Cava, Tilak Raj Singh, James Taggart, Srinivas Suri, and Jason H. Moore. 2019. Learning concise representations for regression by evolving networks of trees. In *Proceedings of the International Conference on Learning Representations* (2019).

[119] William La Cava, Lee Spector, and Kourosh Danai. 2016. Epsilon-lexicase selection for regression. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 741–748.
[120] William G. La Cava, Paul C. Lee, Imran Ajmal, Xiruo Ding, Priyanka Solanki, Jordana B. Cohen, Jason H. Moore, and Daniel S. Herman. 2023. A flexible symbolic regression method for constructing interpretable clinical prediction models. *NPJ Digital Medicine* 6, 1 (2023), 107.
[121] Guillaume Lample and François Charton. 2019. Deep learning for symbolic mathematics. In *Proceedings of the International Conference on Learning Representations*.
[122] Mikel Landajuela, Chak Shing Lee, Jiachen Yang, Ruben Glatt, Claudio P. Santiago, Ignacio Aravena, Terrell Mundhenk, Garrett Mulcahy, and Brenden K. Petersen. 2022. A unified framework for deep symbolic regression. *In Proceedings of the Advances in Neural Information Processing Systems* 35 (2022), 33985–33998.
[123] Nam Le, Hoai Nguyen Xuan, Anthony Brabazon, and Thuong Pham Thi. 2016. Complexity measures in genetic programming learning: A brief review. In *Proceedings of the IEEE Conference on Evolutionary Computation*. 2409–2416.
[124] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. 2011. Genprog: A generic method for automatic software repair. *IEEE Transactions on Software Engineering* 38, 1 (2011), 54–72.
[125] Alessandro Leite and Marc Schoenauer. 2023. Memetic semantic genetic programming for symbolic regression. In *Proceedings of the European Conference on Genetic Programming*. 198–212.
[126] Andrew Lensen, Bing Xue, and Mengjie Zhang. 2020. Genetic programming for evolving a front of interpretable models for data visualization. *IEEE Transactions on Cybernetics* 51, 11 (2020), 5468–5482.
[127] Dongrui Li and Jinghui Zhong. 2020. Dimensionally aware multi-objective genetic programming for automatic crowd behavior modeling. *ACM Transactions on Modeling and Computer Simulation* 30, 3 (2020), 1–24.
[128] Jiachen Li, Ye Yuan, and Hong-Bin Shen. 2022. Symbolic expression transformer: A computer vision approach for symbolic regression. arXiv:2205.11798. Retrieved from https://arxiv.org/abs/2205.11798
[129] Li Li, Minjie Fan, Rishabh Singh, and Patrick Riley. 2019. Neural-guided symbolic regression with asymptotic constraints. arXiv:1901.07714. Retrieved from https://arxiv.org/abs/1901.07714
[130] Wenqiang Li, Weijun Li, Linjun Sun, Min Wu, Lina Yu, Jingyi Liu, Yanjie Li, and Songsong Tian. 2022. Transformer-based model for symbolic regression via joint supervised learning. In *Proceedings of the International Conference on Learning Representations*.
[131] S-C Lin, Georg Martius, and Martin Oettel. 2020. Analytical classical density functionals from an equation learning network. *The Journal of Chemical Physics* 152, 2 (2020), 1–7.
[132] Ziming Liu, Yixuan Wang, Sachin Vaidya, Fabian Ruehle, James Halverson, Marin Soljačić, Thomas Y. Hou, and Max Tegmark. 2024. Kan: Kolmogorov-arnold networks. arXiv:2404.19756. Retrieved from https://arxiv.org/abs/2404.19756
[133] Qiang Lu, Jun Ren, and Zhiguang Wang. 2016. Using genetic programming with prior formula knowledge to solve symbolic regression problem. *Computational Intelligence and Neuroscience* 2016 (2016), 1–1.
[134] José María Luna, Mykola Pechenizkiy, Maria Jose Del Jesus, and Sebastián Ventura. 2017. Mining context-aware association rules using grammar-based genetic programming. *IEEE Transactions on Cybernetics* 48, 11 (2017), 3030–3044.
[135] Yuanzhen Luo, Qiang Lu, Xilei Hu, Jake Luo, and Zhiguang Wang. 2022. Exploring hidden semantics in neural networks with symbolic regression. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 982–990.
[136] Nour Makke and Sanjay Chawla. 2024. Interpretable scientific discovery with symbolic regression: A review. *Artif. Intell. Review* 57, 1 (2024), 2.
[137] Yazmin Maldonado, Ruben Salas, Joel A. Quevedo, Rogelio Valdez, and Leonardo Trujillo. 2024. GSGP-hardware: Instantaneous symbolic regression with an FPGA implementation of geometric semantic genetic programming. *Genet. Program. Evolvable Mach.* 25, 2 (2024), 18.
[138] Abdul Manazir and Khalid Raza. 2019. Recent developments in cartesian genetic programming and its variants. *ACM Computing Surveys* 51, 6 (2019), 1–29.
[139] Joao Francisco BS Martins, Luiz Otavio VB Oliveira, Luis F. Miranda, Felipe Casadei, and Gisele L. Pappa. 2018. Solving the exponential growth of symbolic regression trees in geometric semantic genetic programming. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 1151–1158.
[140] Georg Martius and Christoph H. Lampert. 2016. Extrapolation and learning equations. arXiv:1610.02995. Retrieved from https://arxiv.org/abs/1610.02995
[141] Konstantin T. Matchev, Katia Matcheva, and Alexander Roman. 2022. Analytical modeling of exoplanet transit spectroscopy with dimensional analysis and symbolic regression. *The Astrophysical Journal* 930, 1 (2022), 33.
[142] Yoshitomo Matsubara, Naoya Chiba, Ryo Igarashi, and Yoshitaka Ushiku. 2022. SRSD: Rethinking datasets of symbolic regression for scientific discovery. In *Proceedings of the Advances in Neural Information Processing Systems*.

[143] Trent McConaghy. 2011. FFX: Fast, scalable, deterministic symbolic regression technology. *Genetic Programming Theory and Practice IX* (2011), 235–260.

[144] James McDermott, David R. White, Sean Luke, Luca Manzoni, Mauro Castelli, Leonardo Vanneschi, Wojciech Jaskowski, Krzysztof Krawiec, Robin Harper, Kenneth De Jong, et al. 2012. Genetic programming needs better benchmarks. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 791–798.

[145] Jorge Medina and Andrew D. White. 2023. Active learning in symbolic regression performance with physical constraints. https://doi.org/10.48550/arXiv.2305.10379

[146] Eric Medvet, Marco Virgolin, Mauro Castelli, Peter AN Bosman, Ivo Gonçalves, and Tea Tušar. 2018. Unveiling evolutionary algorithm representation with DU maps. *Genetic Programming and Evolvable Machines* 19 (2018), 351–389.

[147] Yi Mei, Qi Chen, Andrew Lensen, Bing Xue, and Mengjie Zhang. 2022. Explainable artificial intelligence by genetic programming: A survey. *IEEE Transactions on Evolutionary Computation* 27, 3 (2022), 621–641.

[148] Renato Miranda Filho, Anisio Lacerda, and Gisele L. Pappa. 2020. Explaining symbolic regression predictions. In *Proceedings of the IEEE Transactions on Evolutionary Computation* . 1–8.

[149] David J. Montana. 1995. Strongly typed genetic programming. *Evolutionary Computation* 3, 2 (1995), 199–230.

[150] José L. Montaña, César L. Alonso, Cruz E. Borges, and Cristina Tîrnăucă. 2016. Model-driven regularization approach to straight line program genetic programming. *Expert System with Applications* 57 (2016), 76–90.

[151] Alberto Moraglio, Krzysztof Krawiec, and Colin G. Johnson. 2012. Geometric semantic genetic programming. In *Parallel Problem Solving from Nature-PPSN XII: 12th International Conference, Taormina, Italy, September 1-5, 2012, Proceedings, Part I 12*. 21–31.

[152] Flávio AA Motta, João M. De Freitas, Felipe R. De Souza, Heder S. Bernardino, Itamar L. De Oliveira, and Helio JC Barbosa. 2018. A hybrid grammar-based genetic programming for symbolic regression problems. In *Proceedings of the 2018 IEEE Congress on Evolutionary Computation*. 1–8.

[153] T. Nathan Mundhenk, Mikel Landajuela, Ruben Glatt, Claudio P. Santiago, Daniel M. Faissol, and Brenden K. Petersen. 2021. Symbolic regression via neural-guided genetic programming population seeding. arXiv:2111.00053. Retrieved from https://arxiv.org/abs/2111.00053

[154] Luis Muñoz, Leonardo Trujillo, and Sara Silva. 2020. Transfer learning in constructive induction with genetic programming. *Genet. Program. Evolvable Mach.* 21, 4 (2020), 529–569.

[155] Giorgia Nadizar, Luigi Rovito, Andrea De Lorenzo, Eric Medvet, and Marco Virgolin. 2024. An analysis of the ingredients for learning interpretable symbolic regression models with human-in-the-loop and genetic programming. *ACM Transactions on Evolutionary Learning and Optimization* 4, 1 (2024), 1–30.

[156] MZ Naser and Ahmed Z. Naser. 2024. SPINEX_ symbolic regression: Similarity-based symbolic regression with explainable neighbors exploration. arXiv:2411.03358. Retrieved from https://arxiv.org/abs/2411.03358

[157] Pascal Neumann, Liwei Cao, Danilo Russo, Vassilios S. Vassiliadis, and Alexei A. Lapkin. 2020. A new formulation for symbolic regression to identify physico-chemical laws from experimental data. *Chemical Engineering Journal* 387 (2020), 123412.

[158] Isaac Newton. 1934. 1729. The mathematical principles of natural philosophy. *By Sir Isaac Newton. Translated into English by Andrew Motte. To which are added, The laws of the moon's motion, according to gravity. By John Machin Astron. Prof. Gresh. and Secr. R. Soc. In two volumes (Benjamin Motte, at the Middle-Temple-Gate, in Fleetstreet)* 50 (1934).

[159] Quang Uy Nguyen, Xuan Hoai Nguyen, and Michael O'Neill. 2009. Semantic aware crossover for genetic programming: The case for real-valued function regression. In *Proceedings of the European Conference on Genetic Programming*. 292–302.

[160] Su Nguyen, Mengjei Zhang, Damminda Alahakoon, and Kay Chen Tan. 2018. Visualizing the evolution of computer programs for genetic programming [research frontier]. *IEEE Computational Intelligence Magazine* 13, 4 (2018), 77–94.

[161] Ji Ni and Peter Rockett. 2014. Tikhonov regularization as a complexity measure in multiobjective genetic programming. *IEEE Transactions on Evolutionary Computation* 19, 2 (2014), 157–166.

[162] Luiz Otavio VB Oliveira, Fernando EB Otero, and Gisele L. Pappa. 2016. A dispersion operator for geometric semantic genetic programming. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 773–780.

[163] Randal S. Olson, William La Cava, Patryk Orzechowski, Ryan J. Urbanowicz, and Jason H. Moore. 2017. PMLB: A large benchmark suite for machine learning evaluation and comparison. *BioData Mining* 10 (2017), 36.

[164] Michael O'Neill and Conor Ryan. 2001. Grammatical evolution. *IEEE Transactions on Evolutionary Computation* 5, 4 (2001), 349–358.

[165] Michael O'Neill. 2009. Riccardo Poli, William B. Langdon, Nicholas F. McPhee: A Field Guide to Genetic Programming: Lulu. com, 2008, 250 pp, ISBN 978-1-4092-0073-4.

[166] Yangchen Pan, Ehsan Imani, Amir-massoud Farahmand, and Martha White. 2020. An implicit function learning approach for parametric modal regression. *In Proceedings of the Advances in Neural Information Processing Systems* 33 (2020), 11442–11452.

[167] Daniele M. Papetti, Andrea Tangherloni, Davide Farinati, Paolo Cazzaniga, and Leonardo Vanneschi. 2023. Simplifying fitness landscapes using dilation functions evolved with genetic programming. *IEEE Computational Intelligence Magazine* 18, 1 (2023), 22–31.

[168] Tomasz P. Pawlak. 2016. Geometric semantic genetic programming is overkill. In *Genetic Programming: 19th European Conference, EuroGP 2016, Porto, Portugal, March 30-April 1, 2016, Proceedings 19*. 246–260.

[169] Christian S. Perone. 2009. Pyevolve: A python open-source framework for genetic algorithms. *ACM Sigevolution* 4, 1 (2009), 12–20.

[170] Brenden K. Petersen, Mikel Landajuela, T. Nathan Mundhenk, Claudio P. Santiago, Soo K. Kim, and Joanne T. Kim. 2020. Deep symbolic regression: Recovering mathematical expressions from data via risk-seeking policy gradients. (2020). https://arxiv.org/abs/1912.04871

[171] Dao Ngoc Phong, Nguyen Quang Uy, Nguyen Xuan Hoai, and Robert I. McKay. 2012. Evolving approximations for the gaussian q-function by genetic programming with semantic based crossover. In *Proceedings of the IEEE Congress on Evolutionary Computation*. 1–6.

[172] Riccardo Poli, William B. Langdon, Nicholas F. McPhee, and John R. Koza. 2008. A field guide to genetic programming. (2008). https://link.springer.com/article/10.1007/S10710-008-9073-Y

[173] Markus Quade, Julien Gout, and Markus Abel. 2018. Glyph: Symbolic regression tools. arXiv:1803.06226. Retrieved from https://arxiv.org/abs/1803.06226

[174] Atif Rafiq, Enrique Naredo, Meghana Kshirsagar, and Conor Ryan. 2022. On the effect of embedding hierarchy within multi-objective optimization for evolving symbolic regression models. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 594–597.

[175] David L. Randall, Tyler S. Townsend, Jacob D. Hochhalter, and Geoffrey F. Bomarito. 2022. Bingo: A customizable framework for symbolic regression with genetic programming. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 2282–2288.

[176] Shahab Razavi and Eric R. Gamazon. 2022. Neural-network-directed genetic programmer for discovery of governing equations. arXiv:2203.08808. Retrieved from https://arxiv.org/abs/2203.08808

[177] Patrick AK Reinbold, Logan M. Kageorge, Michael F. Schatz, and Roman O. Grigoriev. 2021. Robust learning from noisy, incomplete, high-dimensional experimental data via physically constrained symbolic regression. *Nature Communications* 12, 1 (2021), 3219.

[178] Maximilian Reissmann, Yuan Fang, Andrew Ooi, and Richard Sandberg. 2024. Constraining genetic symbolic regression via semantic backpropagation. arXiv:2409.07369. Retrieved from https://arxiv.org/abs/2409.07369

[179] Subham Sahoo, Christoph Lampert, and Georg Martius. 2018. Learning equations for extrapolation and control. In *Proceedings of the International Conference on Machine Learning*. 4442–4450.

[180] Aliyu Sani Sambo, R. Muhammad Atif Azad, Yevgeniya Kovalchuk, Vivek Padmanaabhan Indramohan, and Hanifa Shah. 2021. Evolving simple and accurate symbolic regression models via asynchronous parallel computing. *Appl. Soft Comput.* 104 (2021), 107198.

[181] Jeffrey R. Sampson. 1976. Adaptation in natural and artificial systems (John H. Holland). https://dl.acm.org/doi/abs/10.1137/1018105

[182] Andrija T. Sarić, Aleksandar A. Sarić, Mark K. Transtrum, and Aleksandar M. Stanković. 2020. Symbolic regression for data-driven dynamic model refinement in power systems. *IEEE Trans. Power Systems* 36, 3 (2020), 2390–2402.

[183] Michael Schmidt and Hod Lipson. 2009. Distilling free-form natural laws from experimental data. *Science* 324, 5923 (2009), 81–85.

[184] Michael Schmidt and Hod Lipson. 2009. Symbolic regression of implicit equations. In *Proceedings of the Genetic Programming Theory and Practice VII*. 73–85.

[185] Eric O. Scott and Sean Luke. 2019. ECJ at 20: Toward a general metaheuristics toolkit. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 1391–1398.

[186] Dominic P. Searson, David E. Leahy, and Mark J. Willis. 2010. GPTIPS: An open source genetic programming toolbox for multigene symbolic regression. In *Proceedings of the Int. Multiconference of Engineers and Computer Scientists Citeseer*. 77–80.

[187] Parshin Shojaee, Kazem Meidani, Amir Barati Farimani, and Chandan Reddy. 2024. Transformer-based planning for symbolic regression. *In Proceedings of the Advances in Neural Information Processing Systems* 36 (2024).

[188] Jinglu Song, Qiang Lu, Bozhou Tian, Jingwen Zhang, Jake Luo, and Zhiguang Wang. 2024. Prove symbolic regression is NP-hard by symbol graph. arXiv:2404.13820. Retrieved from https://arxiv.org/abs/2404.13820

[189] Trevor Stephens. 2019. GPLearn (2015). Retrieved from https://gplearn.readthedocs.io/en/stable/index.html. (2019).

[190] Steven H. Strogatz. 2018. *Nonlinear Dynamics and Chaos with Student Solutions Manual: With Applications to Physics, Biology, Chemistry, and Engineering*. https://doi.org/10.1201/9780429399640

[191] Sheng Sun, Runhai Ouyang, Bochao Zhang, and Tong-Yi Zhang. 2019. Data-driven discovery of formulas by symbolic regression. *MRS Bulletin* 44, 7 (2019), 559–564.

[192] Wassim Tenachi, Rodrigo Ibata, and Foivos I. Diakogiannis. 2023. Deep symbolic regression for physics guided by units constraints: Toward the automated discovery of physical laws. *The Astrophysical Journal* 959, 2 (2023), 99.
[193] Mattias E. Thing and Sofie M. Koksbang. 2024. cp3-bench: A tool for benchmarking symbolic regression algorithms tested with cosmology. arXiv:2406.15531. Retrieved from https://arxiv.org/abs/2406.15531
[194] Yuan Tian, Wenqi Zhou, Hao Dong, David S. Kammer, and Olga Fink. 2024. Sym-Q: Adaptive symbolic regression via sequential decision-making. arXiv:2402.05306. Retrieved from https://arxiv.org/abs/2402.05306
[195] Ho Fung Tsoi, Vladimir Loncar, Sridhara Dasu, and Philip Harris. 2024. SymbolNet: Neural symbolic regression with adaptive dynamic pruning. arXiv:2401.09949. Retrieved from https://arxiv.org/abs/2401.09949
[196] Silviu-Marian Udrescu and Max Tegmark. 2020. AI feynman: A physics-inspired method for symbolic regression. *Science Advances* 6, 16 (2020), eaay2631.
[197] Silviu-Marian Udrescu and Max Tegmark. 2021. Symbolic pregression: Discovering physical laws from distorted video. *Physical Review E* 103, 4 (2021), 043307.
[198] Nguyen Quang Uy, Nguyen Xuan Hoai, Michael O'Neill, Robert I. McKay, and Edgar Galván-López. 2011. Semantically-based crossover in genetic programming: Application to real-valued symbolic regression. *Genetic Programming and Evolvable Machines* 12 (2011), 91–119.
[199] Nguyen Quang Uy, Nguyen Xuan Hoai, Michael O'Neill, Robert I. McKay, and Dao Ngoc Phong. 2013. On the roles of semantic locality of crossover in genetic programming. *Information Sciences* 235 (2013), 195–213.
[200] Harsha Vaddireddy and Omer San. 2019. Equation discovery using fast function extraction: A deterministic symbolic regression approach. *Fluids* 4, 2 (2019), 111.
[201] Mojtaba Valipour, Bowen You, Maysum Panju, and Ali Ghodsi. 2021. Symbolicgpt: A generative transformer model for symbolic regression. arXiv:2106.14131. Retrieved from https://arxiv.org/abs/2106.14131
[202] Jan N. Van Rijn, Bernd Bischl, Luis Torgo, Bo Gao, Venkatesh Umaashankar, Simon Fischer, Patrick Winter, Bernd Wiswedel, Michael R. Berthold, and Joaquin Vanschoren. 2013. OpenML: A collaborative science platform. In *Proceedings of the Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. 645–649.
[203] Leonardo Vanneschi and Mauro Castelli. 2021. Soft target and functional complexity reduction: A hybrid regularization method for genetic programming. *Expert Systems with Applications* 177 (2021), 114929.
[204] Leonardo Vanneschi, Mauro Castelli, and Sara Silva. 2014. A survey of semantic methods in genetic programming. *Genetic Programming and Evolvable Machines* 15 (2014), 195–214.
[205] Marco Virgolin, Tanja Alderliesten, and Peter AN Bosman. 2019. Linear scaling with and within semantic backpropagation-based genetic programming for symbolic regression. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 1084–1092.
[206] Marco Virgolin, Tanja Alderliesten, Cees Witteveen, and Peter AN Bosman. 2021. Improving model-based genetic programming for symbolic regression of small expressions. *Evolutionary Computation* 29, 2 (2021), 211–237.
[207] Marco Virgolin and Peter AN Bosman. 2022. Coefficient mutation in the gene-pool optimal mixing evolutionary algorithm for symbolic regression. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 2289–2297.
[208] Marco Virgolin and Solon P. Pissis. 2022. Symbolic regression is np-hard. *Transactions on Machine Learning Research* (2022). https://doi.org/10.48550/arXiv.2207.01018
[209] Ekaterina J. Vladislavleva, Guido F. Smits, and Dick Den Hertog. 2008. Order of nonlinearity as a complexity measure for models generated by symbolic regression via pareto genetic programming. *IEEE Transactions on Evolutionary Computation* 13, 2 (2008), 333–349.
[210] Yiqun Wang, Nicholas Wagner, and James M. Rondinelli. 2019. Symbolic regression in materials science. *MRS Communications* 9, 3 (2019), 793–805.
[211] Jiahao Wen, Junlan Dong, and Jinghui Zhong. 2024. Sign change detection based fitness evaluation for automatic implicit equation discovery. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 980–987.
[212] Matthias Werner, Andrej Junginger, Philipp Hennig, and Georg Martius. 2021. Informed equation learning. arXiv:2105.06331. Retrieved from https://arxiv.org/abs/2105.06331
[213] Sebastian Johann Wetzel. 2024. Closed-form interpretation of neural network classifiers with symbolic regression gradients. arXiv:2401.04978. Retrieved from https://arxiv.org/abs/2401.04978
[214] Edward William and James Northern. 2008. Genetic programming lab (GPLab) tool set version 3.0. In *Proceedings of the IEEE Region 5 Conference*. 1–6.
[215] Casper Wilstrup and Jaan Kasak. 2021. Symbolic regression outperforms other models for small data sets. arXiv:2103.15147. Retrieved from https://arxiv.org/abs/2103.15147
[216] David Wittenberg, Franz Rothlauf, and Dirk Schweim. 2020. DAE-GP: Denoising autoencoder LSTM networks as probabilistic models in estimation of distribution genetic programming. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 1037–1045.

[217] Tony Worm and Kenneth Chiu. 2013. Prioritized grammar enumeration: Symbolic regression by dynamic programming. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 1021–1028.
[218] Min Wu, Weijun Li, Lina Yu, Wenqiang Li, Jingyi Liu, Yanjie Li, and Meilan Hao. 2024. PruneSymNet: A symbolic neural network and pruning algorithm for symbolic regression. arXiv:2401.15103. Retrieved from https://arxiv.org/abs/2401.15103
[219] Min Wu, Weijun Li, Lina Yu, Linjun Sun, Jingyi Liu, and Wenqiang Li. 2025. Discovering mathematical expressions through DeepSymNet: A classification-based symbolic regression framework. *IEEE Transactions on Neural Networks and Learning Systems* 36, 1 (2025), 1356–1370.
[220] Hengrui Xing, Ansaf Salleb-Aouissi, and Nakul Verma. 2021. Automated symbolic law discovery: A computer vision approach. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 660–668.
[221] Kunpeng Xu, Lifei Chen, and Shengrui Wang. 2024. Kolmogorov-arnold networks for time series: Bridging predictive power and interpretability. arXiv:2406.02496. Retrieved from https://arxiv.org/abs/2406.02496
[222] Kaifeng Yang and Michael Affenzeller. 2023. Surrogate-assisted multi-objective optimization via genetic programming based symbolic regression. In *Proceedings of the International Conference on Evolutionary Multi-criterion Optimization*. 176–190.
[223] Mengjiao Yang and Been Kim. 2019. BIM: Towards quantitative evaluation of interpretability methods with ground truth. arXiv:1907.09701. Retrieved from https://arxiv.org/abs/1907.09701
[224] Yifan Yang, Gang Chen, Hui Ma, Sven Hartmann, and Mengjie Zhang. 2024. Dual-tree genetic programming with adaptive mutation for dynamic workflow scheduling in cloud computing. *IEEE Transactions on Evolutionary Computation* (2024).
[225] Jan Žegklitz and Petr Pošík. 2021. Benchmarking state-of-the-art symbolic regression algorithms. *Genetic Programming and Evolvable Machines* 22, 1 (2021), 5–33.
[226] Ruihua Zeng, Zhixing Huang, Yongliang Chen, Jinghui Zhong, and Liang Feng. 2020. Comparison of different computing platforms for implementing parallel genetic programming. In *Proceedings of the IEEE Congress on Evolutionary Computation*. 1–8.
[227] Hengzhe Zhang, Qi Chen, Bing Xue, Wolfgang Banzhaf, and Mengjie Zhang. 2023. A double lexicase selection operator for bloat control in evolutionary feature construction for regression. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 1194–1202.
[228] Hengzhe Zhang and Aimin Zhou. 2021. RL-GEP: Symbolic regression via gene expression programming and reinforcement learning. In *Proceedings of the 2021 International Joint Conference on Neural Networks*. 1–8.
[229] Michael Zhang, Samuel Kim, Peter Y. Lu, and Marin Soljačić. 2023. Deep learning and symbolic regression for discovering parametric equations. *IEEE Transactions on Neural Networks and Learning Systems* 35, 11(2024), 16775–16787.
[230] Rui Zhang, Andrew Lensen, and Yanan Sun. 2022. Speeding up genetic programming based symbolic regression using GPUs. In *Proceedings of the Pacific Rim International Conference on Artificial Intelligence*. 519–533.
[231] Zhixin Zhang and Zhiyong Chen. 2021. Modeling and control of robotic manipulators based on symbolic regression. *IEEE Transactions on Neural Networks and Learning Systems* 34, 5 (2021), 2440–2450.
[232] Wenqing Zheng, Tianlong Chen, Ting-Kuei Hu, and Zhangyang Wang. 2022. Symbolic learning to optimize: Towards interpretability and scalability. arXiv:2203.06578. Retrieved from https://arxiv.org/abs/2203.06578
[233] Jinghui Zhong, Wentong Cai, Michael Lees, and Linbo Luo. 2017. Automatic model construction for the behavior of human crowds. *Applied Soft Computing* 56 (2017), 368–378.
[234] Jinghui Zhong, Junlan Dong, Wei-Li Liu, Liang Feng, and Jun Zhang. 2025. Multiform genetic programming framework for symbolic regression problems. *IEEE Transactions on Evolutionary Computation* 29, 2 (2025), 429–443.
[235] Jinghui Zhong, Liang Feng, and Yew-Soon Ong. 2017. Gene expression programming: A survey. *IEEE Computational Intelligence Magazine* 12, 3 (2017), 54–72.
[236] Jinghui Zhong, Yusen Lin, Chengyu Lu, and Zhixing Huang. 2018. A deep learning assisted gene expression programming framework for symbolic regression problems. In *Proceedings of the Neural Information Processing: 25th International Conference*. 530–541.
[237] Jinghui Zhong, Yew-Soon Ong, and Wentong Cai. 2015. Self-learning gene expression programming. *IEEE Transactions on Evolutionary Computation* 20, 1 (2015), 65–80.
[238] Jinghui Zhong, Jiaquan Yang, Yongliang Chen, Wei-Li Liu, and Liang Feng. 2021. Mining implicit equations from data using gene expression programming. *IEEE Transactions on Emerging Topics in Computing* 10, 2 (2021), 1058–1074.
[239] Lianjie Zhong, Jinghui Zhong, and Chengyu Lu. 2021. A comparative analysis of dimensionality reduction methods for genetic programming to solve high-dimensional symbolic regression problems. In *Proceedings of the IEEE International Conference on Systems, Man and Cybernetics* 476–483.
[240] Hongpeng Zhou and Wei Pan. 2022. Bayesian learning to discover mathematical operations in governing equations of dynamic systems. arXiv:2206.00669. Retrieved from https://arxiv.org/abs/2206.00669

Appendix

Links

AI Feynman dataset: https://space.mit.edu/home/tegmark/aifeynman.html
SRSD: https://huggingface.co/yoshitomo-matsubara
Storgatz dataset: https://williamlacava.com/ode-strogatz/
PMLB: https://github.com/EpistasisLab/pmlb
UCI datasetS: https://archive.ics.uci.edu/ml/index.php
OpenML datasets: https://github.com/EpistasisLab/pmlb/tree/master/datasets
GE Link: https://www.grammatical-evolution.org
SL-GEP Link: https://jinghuizhong.com/wp-content/uploads/2019/08/sl-gep-code-dataset.zip
MLSI Link: https://github.com/Zhixing1020/Linear-Genetic-Programming-LGP-and-Applications
DLS_GP Link: https://github.com/hengzhe-zhang/DoubleLexicaseSelection
PLS Link: https://github.com/ld-ing/plexicase
MSGP Link: https://gitlab.inria.fr/trust-ai/memetics/msgp
GSGP Link: https://github.com/amoraglio/GSGP
GSGP-Red Link: https://github.com/laic-ufmg/GSGP-Red
SBP-GP Link: https://github.com/marcovirgolin/GP-GOMEA
SLGP Link: https://github.com/Zhixing1020/SemanticLGP_for_SR
GP/TS-S Link: https://github.com/chuthihuong/GP
SciMED Link: https://github.com/LironSimon/SciMED
ELA Link: https://github.com/renatomir/ELA-WCCI2020
DU Map Link: https://machinelearning.inginf.units.it/data-and-tools/unveiling-evolutionary-algorithm-representation-with-du-maps
DSR link: https://github.com/dso-org/deep-symbolic-optimization
NGPPS link: https://github.com/dso-org/deep-symbolic-optimization
RBG2-SR link: https://github.com/laure-crochepierre/reinforcement-based-grammar-guided-symbolic-regression
IRL-SR link: https://github.com/laure-crochepierre/interactive-rbg2sr
Phi-SO link: https://github.com/WassimTenachi/PhySO
Sym-Q link: https://github.com/Wenqi-Z/Sym-Q
Sy-Mathematics link: https://github.com/facebookresearch/SymbolicMathematics
SymbolicGPT link: https://github.com/mojivalipour/symbolicgpt
NeSymReS link: https://github.com/SymposiumOrganization/NeuralSymbolicRegressionThatScales
E2E link: https://github.com/facebookresearch/symbolicregression
JSL-SR link: https://github.com/AILWQ/Joint_Supervised_Learning_for_SR
ODEforme link: https://github.com/sdascoli/odeformer
DGSR link: https://github.com/samholt/DeepGenerativeSymbolicRegression
TPSR link: https://github.com/deep-symbolic-mathematics/TPSR
EQL link: https://github.com/KristofPusztai/EQL
EQL$^{\div}$ link: https://github.com/martius-lab/EQL
IEQL link: https://github.com/samuelkim314/DeepSymReg
GMEQL link: https://github.com/aaron-vuw/gmeql
FEQL link: https://github.com/ShangChunLin/FEQL
OccamNet link: https://github.com/druidowm/OccamNet_Public
MathONet link: https://github.com/nips2021anonymous/MathONet

Parametric-EQL link: https://github.com/samuelkim314/parametric-eql
DeepSymNet link: https://github.com/wumin86/DeepSymNet
PruneSymNet link: https://github.com/wumin86/PruneSymNet
SymbolNet link: https://github.com/hftsoi/SymbolNet
GrammarVAE link: https://github.com/mkusner/grammarVAE
GNSR link: https://github.com/MilesCranmer/symbolic_deep_learning
AI-Feynman link: https://github.com/SJ001/AI-Feynman
EQDiscovery link: https://github.com/isds-neu/EQDiscovery
NSRwH link: https://github.com/SymposiumOrganization/ControllableNeuralSymbolicRegression
SISR link: https://github.com/dandip/SISR
KAN link: https://github.com/KindXiaoming/pykan
Operon link: https://github.com/heal-research/pyoperon
SBP-GP link: https://github.com/marcovirgolin/GP-GOMEA
uDSR link: https://github.com/dso-org/deep-symbolic-optimization
Bingo link: https://github.com/nasa/bingo
E2ET link: https://github.com/pakamienny/e2e_transformer
RILS-ROLS link: https://github.com/kartelj/rils-rols
MRGP link: https://github.com/flexgp/gp-learners
GP-GOMEA link: https://github.com/marcovirgolin/GP-GOMEA
FEAT link: https://github.com/cavalab/feat
SRBench link: https://github.com/cavalab/srbench
SRBench++ link: https://cavalab.org/srbench/competition-2022/
CP3-bench link: https://github.com/CP3-Origins/cp3-bench
iirsBenchmark link: https://github.com/gAldeia/iirsBenchmark
GPLab link: https://gplab.sourceforge.net/
GenProg link: https://github.com/squaresLab/genprog-code
GPC++ link: http://www0.cs.ucl.ac.uk/staff/W.Langdon/ftp/weinbenner/gp.html
ellenGP link: https://github.com/lacava/ellen
GPTIPS link: https://sites.google.com/site/gptips4matlab/
PonyGE2 link: https://github.com/jmmcd/PonyGE2
WebGE link: https://github.com/GRAFO-URJC/WebGE
Eureqa link: http://nutonian.wikidot.com/
QLattice link: https://github.com/abzu-ai/QLattice-clinical-omics
Glyph link: https://github.com/Ambrosys/glyph
DEAP link: https://github.com/DEAP/deap
GPlearn link: https://github.com/trevorstephens/gplearn
PySR link: https://github.com/MilesCranmer/PySR
HeuristicLab link: https://dev.heuristiclab.com/trac.fcgi/
Data-Modeler link: https://evolved-analytics.com/
ALAMO link: https://minlp.com/alamo-modeling-tool
HyperGP link: https://github.com/MZT-srcount/HyperGP

Received 13 July 2024; revised 15 April 2025; accepted 29 April 2025